ENTERPRISE PLATFORM ARCHITECTURE

AeroBook

Cloud Airline Reservation & Operations Ecosystem

Complete Technical Architecture, Microservices Directory, REST API Catalog, Relational Data Persistence, Security Specifications, and Operational Runbooks.

Java 21 LTS | Spring Boot 3.3.4 | Angular 18.2 | Spring Cloud Gateway | Netflix Eureka | MySQL 8.0 | RabbitMQ AMQP | Razorpay Payment Gateway | SonarQube Governed

Author: AeroBook Platform Engineering & Architecture Team
Document Status: Production Verified | Version 1.0.0
Date: October 2026
Target Environment: Cloud Native / Microservices Monorepo

Table of Contents

1 Project Overview & Executive Summary

2 Technology Stack & Version Matrix

3 System Architecture & Boundaries

4 Project Structure & Monorepo Anatomy

5.1 Microservice: Service Discovery (Eureka)

5.2 Microservice: Config Server

5.3 Microservice: API Gateway

5.4 Microservice: Auth Service

5.5 Microservice: User Service

5.6 Microservice: Flight Service

5.7 Microservice: Fare Service

5.8 Microservice: Booking Service

5.9 Microservice: Check-In Service

5.10 Microservice: Notification Service

5.11 Microservice: Payment Service

6 Frontend Architecture (Angular 18)

7 Master REST API Reference Catalog

8 Database Design & Relational Persistence

9 Inter-Service Communication (Feign & RabbitMQ)

10 Platform Security, JWT & RBAC Matrix

11 Configuration Reference & Environment Catalog

12 Local Development & Startup Guide

13 Deployment & Containerization Architecture

14 Testing & Quality Assurance Architecture

15 Observability, Monitoring & Logging

16 Core Business Workflows & Sequence Flows

17 Operational Troubleshooting Runbook

18 Domain & Technical Glossary

19 Architecture Decision Records (ADRs)

20 Architecture Evolution & Recommendations

21 Definitive Platform Service Inventory

22 Documentation Coverage & Verification Audit

CHAPTER 1

01. Project Overview & Business Domain

1. Executive Summary

AeroBook is an enterprise-grade, cloud-native airline flight reservation, passenger services, airport operations, and fleet management platform. Designed around modern distributed microservices principles, AeroBook provides a seamless, real-time end-to-end journey for airline passengers, airport ground staff, and flight operations administrators.

The application spans **11 Java Spring Boot backend microservices**, **1 Angular 18 Single Page Application (SPA)**, and **8 decoupled MySQL database schemas**, tied together through a reactive **Spring Cloud API Gateway** with client-side load balancing via **Netflix Eureka Service Discovery**.

---

2. Business Problem & Objectives

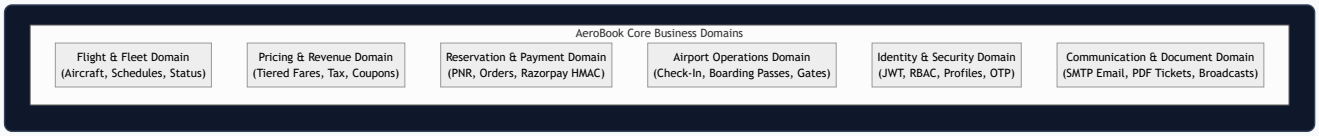
Modern commercial aviation demands robust, highly resilient software architectures capable of handling concurrent flight searches, dynamic fare pricing, high-volume transactional booking reservations, secure cryptographic payment processing, and instantaneous airport ground operations.

Key Business Challenges Addressed:

1. **Fault Isolation**: Failure in ancillary services (such as notification dispatch or incident reporting) must never prevent a passenger from booking a flight or boarding an aircraft.
2. **Scalability & Decoupling**: High-traffic passenger searches during holiday travel surges require independent horizontal scaling without overloading reservation or payment databases.
3. **Data Integrity & Consistency**: Booking flight seats, processing payments, and issuing passenger name records (PNRs) must maintain strict consistency across distributed services.
4. **Multi-Persona Airport Experience**: Distinct personas (Passengers, Ground Operations Staff, Airline Administrators) require specialized interfaces, strict role-based access control (RBAC), and tailored workflows.
5. **Regulatory Compliance & Standards**: Generating authentic IATA Bar Coded Boarding Passes (BCBP), formal GST/tax invoices, and meeting WCAG AA accessibility standards for digital passenger channels.

---

3. Core Domains & Features



3.1 Customer / Passenger Domain

- **Flight Discovery**: Real-time origin/destination search, departure date filtering, available seat counter, and flight radar previews.
- **Tiered Cabin Selection**: Transparent pricing across Economy, Business, and First Class cabins with automated tax breakdown (PSF, UDF, CGST, SGST).
- **Promotional Discounts**: Coupon engine supporting percentage discounts with minimum fare criteria and maximum caps (e.g., `AERO2026`, `WELCOME10`).
- **Reservation & PNR**: Instant reservation generation with unique 8-character alphanumeric Passenger Name Record (PNR).
- **Payment Processing**: Integrated Razorpay modal checkout supporting mock simulations, live test tokens, and server-side HMAC-SHA256 signature verification.
- **Digital E-Ticket & Tax Invoice**: High-fidelity dual-view modal, A4-optimized print styles, OpenPDF document download, and email delivery.
- **Web Check-In & Boarding Pass**: Interactive cabin fuselage seat map selection, automated barcode generation (`BP-X-YYZ`), and digital boarding pass retrieval.
- **Passenger Loyalty & Gamification**: Real-time XP tracking, traveler rank tiers (*Bronze Wanderer* to *Diamond Elite*), active streak calculations, and achievement badges.
- **Real-Time Passenger Alerts**: Automated notifications for booking confirmations, gate changes, flight delays, and weather advisories.

3.2 Airport Ground Operations (Staff) Domain

- **Passenger Check-In Desk**: Quick passenger search by PNR, seat reassignment, and manual check-in processing.
- **Departure Gate Boarding Scanner**: Real-time barcode validation updating passenger status from `CHECKED_IN` to `BOARDED`.
- **Flight Passenger Manifest**: Real-time visibility into passenger manifests, checked baggage tags, and boarding completion rates.
- **Flight Status Dispatcher**: Operational status updates (`SCHEDULED`, `BOARDING`, `DELAYED`, `DEPARTED`, `CANCELLED`).
- **Airport Incident Reporting**: Ground staff ticketing system for wheelchair assistance, baggage claims, transit advisories, and medical needs.

3.3 Airline Administration Domain

- **Commercial Flight Scheduling**: Route setup, departure/arrival timestamps, aircraft linkage, base fare assignment, and capacity limits.
- **Fleet Inventory Management**: Aircraft registration, manufacturer tracking (Airbus, Boeing, ATR), and cabin seating capacity.
- **Revenue & Dynamic Pricing**: Cabin-class multipliers, tax rules, and seasonal pricing adjustments.
- **Coupon & Campaign Management**: Promotional code creation, usage quotas, active/inactive toggling, and date validity.

- **User & Credential Directory**: Cross-platform customer audit, account verification state, and administrative role promotions.
- **System Diagnostics & Monitoring**: Gateway health probes, route registries, and SonarQube code quality dashboards.

---

4. User Personas & Access Control

AeroBook strictly enforces three authenticated roles and one unauthenticated public persona:

Persona	Role Identifier	Access Privileges	Primary Interface
Guest / Visitor	*Unauthenticated*	Flight search, schedule browsing, coupon validation, email verification, login/registration	Home Landing Page (`/`)
Passenger	`ROLE_USER`	Personal flight bookings, payment checkout, web check-in, seat selection, PDF downloads, notifications, loyalty profile	Customer Dashboard (`/customer`)
Ground Staff	`ROLE_STAFF`	Airport check-in desk, boarding pass scanner, passenger manifest oversight, flight status dispatching, incident logging	Staff Dashboard (`/staff`)
System Admin	`ROLE_ADMIN`	Flight schedule creation, aircraft fleet management, fare pricing, coupon administration, credential audit, user management	Admin Dashboard (`/admin`)

---

5. Functional & Non-Functional Requirements

Functional Requirements

- **FR-01 (Authentication)**: Secure registration with BCrypt password hashing, JWT access token generation (24-hour expiration), and refresh token rotation.
- **FR-02 (Role Routing)**: Post-login role inspection automatically routing passengers to `/customer`, ground crew to `/staff`, and administrators to `/admin`.
- **FR-03 (Inventory Synchronization)**: Booking reservations must decrement available seats in real time and reject overbooking.
- **FR-04 (Cryptographic Verification)**: Razorpay payment signatures must be calculated using HMAC-SHA256 and verified prior to marking transactions `CAPTURED`.
- **FR-05 (Automated Document Generation)**: E-Tickets and Boarding Passes must be dynamically generated as vector PDFs using OpenPDF with embedded barcodes.
- **FR-06 (Asynchronous Messaging)**: Confirmed bookings must emit `BOOKING_CREATED` events to RabbitMQ for flight and notification consumers.

Non-Functional Requirements

- **NFR-01 (Resilience & Decoupling)**: Microservices must maintain isolated data stores (Database-per-Service) and graceful fallback when ancillary services are offline.
- **NFR-02 (Performance & Latency)**: Sub-second response times for flight searches via non-blocking reactive API Gateway routing.
- **NFR-03 (Accessibility - WCAG AA)**: Frontend contrast ratios must exceed 4.5:1 for normal text and 3:1 for graphical elements across both light and dark themes.
- **NFR-04 (Code Quality & Security)**: Zero high-severity vulnerabilities, JaCoCo XML backend test coverage reporting, and SonarQube quality gate integration.
- **NFR-05 (Zero Theme Flicker)**: Immediate pre-paint execution of theme preferences in browser DOM to avoid theme flashing.

---

6. Implementation Status & Limitations

Current Implementation Status

- **Backend Microservices (11/11)**: Fully functional, compiling, tested, and integrated with JaCoCo and SonarQube Maven plugins.
- **Frontend SPA (1/1)**: Fully functional Angular 18 standalone component architecture with 0 mock dependencies, WCAG AA compliance, and automated Karma/ChromeHeadless tests.
- **Database Layer (8/8)**: Schema creation scripts and full master seed data (`scripts/seed.sql`) covering 20 users, 16 aircraft, 25 flights, 25 fares, 16 coupons, 22 bookings, 18 check-ins, 22 payments, and 22 notifications.
- **Quality Analysis**: Complete SonarQube configuration and master automation scripts (`run-sonar-analysis.ps1` , `run-sonar-analysis.bat`).

Known Operational Limitations

1. **Localhost Routing**: Microservices currently default to `localhost` ports; Kubernetes manifest definitions and cloud container registries are planned for future deployment phases.
2. **RabbitMQ Optional Mode**: RabbitMQ messaging defaults to disabled in development environments lacking a local broker, falling back to direct synchronous REST calls.
3. **Razorpay Live Gateway**: Razorpay operates in test/mock mode using cryptographic simulation tokens unless configured with live merchant API keys in `.env`.

CHAPTER 2

02. Complete Technology Stack & Version Matrix

1. Technology Inventory Overview

AeroBook is constructed using modern enterprise software engineering tools. Every technology, framework, library, and runtime documented below has been verified directly against source code files, `pom.xml` manifests, `package.json` specifications, and runtime log output.

2. Core Runtimes & Languages

Technology	Verified Version	Purpose in AeroBook	Repository Locations
Java Development Kit (JDK)	**17 (LTS)** *(Java 20 runtime compatible)*	Primary programming language for all 11 backend microservices	`*/pom.xml` (`<java.version>17</java.version>`)
Node.js	**v22.17.0**	Runtime environment for Angular build tools, CLI, and test runners	`frontend/package.json`
TypeScript	**~5.5.2**	Strictly typed frontend language for Angular components, models, and APIs	`frontend/package.json`, `frontend/tsconfig.json`
SQL	**MySQL 8.x dialect**	Relational data definition and query language for 8 microservice databases	`scripts/seed.sql`, `*/application.properties`
PowerShell	**5.1 / 7+**	Automation scripting for multi-service orchestration, seeding, and SonarQube	`scripts/*.ps1`, `run-sonar-analysis.ps1`
Windows Shell (CMD)	**Windows 10/11**	Quick-launch batch runners for developers without PowerShell policy adjustments	`*.bat`, `run-sonar-analysis.bat`

3. Backend Microservices Framework & Dependencies



3.1 Spring Boot Ecosystem

Framework / Module	Version	Purpose & Architectural Role	Services Utilizing
Spring Boot Parent	**4.1.1**	Core backend microservice framework providing auto-configuration, dependency management, and embedded Tomcat	`service-discovery`, `config-server`, `auth-service`, `user-service`, `fare-service`, `booking-service`, `check-in-service`, `notification-service`, `payment-service`
Spring Boot Parent	**3.5.5**	Core framework for non-blocking reactive gateway and flight scheduling services	`api-gateway`, `flight-service`
Spring Cloud Dependencies	**2025.1.3 / 2025.0.0**	Cloud coordination suite managing service discovery, client routing, and configuration import	All 11 microservices
Spring Cloud Gateway WebFlux	**2025.0.0 (Reactive)**	High-throughput, non-blocking asynchronous reverse proxy, route routing, and JWT filter pipeline	`api-gateway`
Netflix Eureka Server	**2025.1.3**	High-availability central service registry and heartbeat monitoring	`service-discovery`
Netflix Eureka Client	**2025.1.3 / 2025.0.0**	Automatic service registration and client-side load balancing (`lb://`)	All 10 client services
Spring Cloud OpenFeign	**2025.1.3 / 2025.0.0**	Declarative REST client interfaces for synchronous inter-service communication	`auth-service`, `booking-service`, `fare-service`, `flight-service`
Spring Cloud Config Server	**2025.1.3**	Optional centralized environment configuration repository	`config-server`

3.2 Data Persistence, Messaging & Ancillary Libraries

Library	Version	Purpose	Component
Spring Data JPA	*Spring Boot BOM*	Relational data repository abstraction and CRUD lifecycle management	8 persistence services

Library	Version	Purpose	Component
Hibernate ORM	**7.4.5.Final**	Object-relational mapping (ORM) provider implementing JPA specifications	8 persistence services
MySQL Connector/J	**8.x / 9.x**	Official JDBC driver connecting Spring Data JPA to MySQL instances	8 persistence services
Spring AMQP / RabbitMQ	*Spring Boot BOM*	Asynchronous message broker abstraction for event publication and consumption	`booking-service` , `flight-service`
JJWT (Java JWT)	**0.12.7**	Cryptographic HMAC-SHA signing, parsing, and validation of JWT access/refresh tokens	`api-gateway` , `auth-service`
OpenPDF	**2.0.3**	Airline-grade vector PDF generation for E-Tickets, Tax Invoices, and Boarding Passes	`notification-service`
Spring Boot Starter Mail	*Spring Boot BOM*	SMTP email dispatch supporting MIME multipart HTML templates	`notification-service`
ModelMapper	**3.2.1**	Java Bean mapping between JPA Entities and REST DTOs	`flight-service`
Project Lombok	*Spring Boot BOM*	Compile-time bytecode generation for getters, setters, builders, and constructors	`api-gateway` , `auth-service` , `booking-service` , `flight-service`
Spring Boot Actuator	*Spring Boot BOM*	Operational production monitoring, metrics, and health probes (`/actuator/health`)	`service-discovery` , `config-server` , `api-gateway` , `booking-service`

4. Frontend Single Page Application (SPA) Stack



4.1 UI Framework & State Management

Package	Version	Purpose & Role in AeroBook Frontend
@angular/core	**^18.2.0**	Modern Angular framework utilizing Standalone Components and Signals
@angular/common	**^18.2.0**	Common pipes (`date` , `currency` , `uppercase`), directives, and formatting
@angular/router	**^18.2.0**	Client-side routing, route guards (`customerGuard` , `staffGuard` , `adminGuard`)
@angular/forms	**^18.2.0**	Reactive forms and template-driven form validation
@angular/platform-browser	**^18.2.0**	DOM rendering engine, browser platform initialization
@angular/animations	**^18.2.0**	Smooth UI micro-interactions, modal transitions, and accordion animations
rxjs	**~7.8.0**	Reactive extensions handling HTTP interceptors, polling streams, and event buses
zone.js	**~0.14.10**	Angular change detection execution context
tslib	**^2.3.0**	TypeScript runtime helper library

4.2 Styling, Typography & Design System

Package	Version	Purpose & Role
tailwindcss	**^3.4.17**	Utility-first CSS framework configured with high-contrast dual-theme tokens
postcss	**^8.5.28**	CSS transformation and Tailwind build pipeline processor
autoprefixer	**^10.6.1**	Vendor prefixing for cross-browser CSS support
@lucide/angular / lucide-angular	**^1.48.0**	Aviation and dashboard iconography (airplane, calendar, shield, user, etc.)
@fontsource/plus-jakarta-sans	**^5.3.0**	High-legibility geometric sans-serif corporate typography

5. Quality Assurance, Testing & Static Analysis Stack

Tool / Plugin	Version	Role in Pipeline	Scope
JUnit Jupiter (JUnit 5)	**5.10.x**	Unit and integration test runner for backend services	All 11 microservices
Mockito	**5.x**	Mocking framework for unit testing service layers without live DBs	All 11 microservices

Tool / Plugin	Version	Role in Pipeline	Scope
Spring Boot Starter Test	*BOM*	Integration test harness with MockMvc and Spring context testing	All 11 microservices
Reactor Test	*BOM*	StepVerifier testing for reactive Gateway WebFlux pipelines	`api-gateway`
JaCoCo Maven Plugin	**0.8.12**	Java code coverage instrumentation producing XML execution reports	All 11 microservices
SonarQube Scanner for Maven	**5.0.0.4389**	Maven plugin analyzing Java source code, rulesets, and JaCoCo reports	All 11 microservices
Karma	**~6.4.0**	Test runner executing Jasmine test specs in real browser environments	`frontend`
Jasmine Core	**~5.2.0**	Behavior-driven development testing framework for JavaScript/TypeScript	`frontend`
karma-chrome-launcher	**~3.2.0**	Headless Chrome browser integration for Karma	`frontend`
karma-coverage	**~2.2.0**	Istanbul/LCOV code coverage reporter for Angular	`frontend`
SonarScanner for NPM (@sonar/scan)	**5.0.2**	TypeScript and Angular static analysis scanner importing LCOV reports	`frontend`
SonarQube Server (Community)	**10.5**	Centralized code quality, security vulnerability, and bug analysis server	Root Platform
PostgreSQL	**15-alpine**	Dedicated relational backend datastore for SonarQube Server	`docker-compose.sonarqube.yml`

6. Build Systems, Packaging & Package Managers

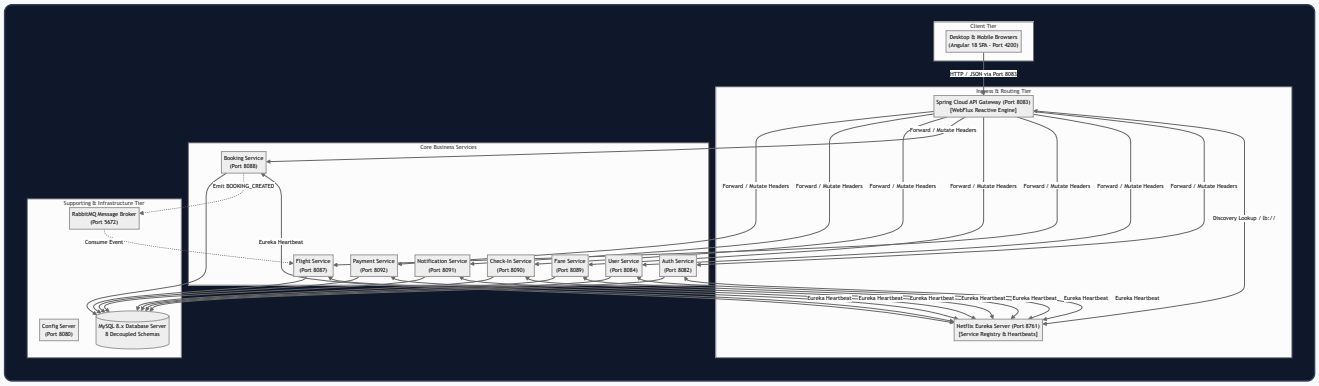
Build Tool	Version	Configuration Files	Purpose
Apache Maven	**3.9.16** *(via Maven Wrapper `mvnw`)*	`pom.xml`, `*/pom.xml`, `.mvn/wrapper/maven-wrapper.properties`	Root aggregator compilation, testing, packaging (`jar` / `war`), and SonarQube scan
Angular CLI	**^18.2.21**	`frontend/angular.json`, `frontend/tsconfig.json`	Single command SPA building, watching, and Karma test launching
npm	**10.9.2**	`frontend/package.json`, `frontend/package-lock.json`	Dependency resolution, package locking, and frontend script automation
Docker Compose	**v2.x compatible**	`docker-compose.sonarqube.yml`	Containerized SonarQube + PostgreSQL service orchestration

CHAPTER 3

03. System Architecture & Design Principles

1. Architectural Philosophy & Core Patterns

AeroBook is architectured according to cloud-native microservices patterns to deliver high availability, fault isolation, independent deployability, and maintainability.



Key Architectural Patterns Implemented:

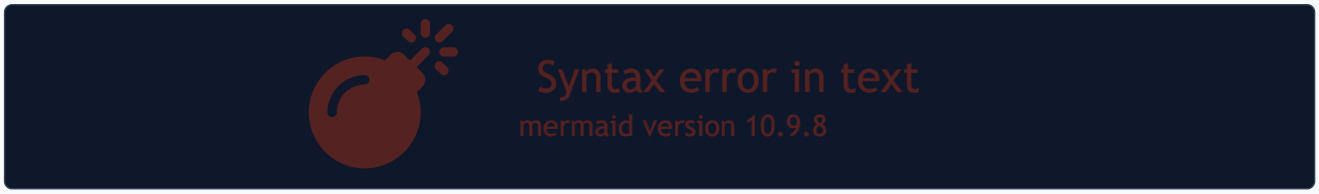
1. **API Gateway Pattern**: Single unified ingress point (`http://localhost:8083`) shielding internal microservice network topology, terminating CORS, validating JWTs, and tracing requests.
2. **Service Discovery Pattern**: Centralized Netflix Eureka registry (`http://localhost:8761`). Services register dynamically upon boot; the Gateway uses `lb://SERVICE-NAME` for reactive client-side load balancing.
3. **Database-per-Service Pattern**: Each business microservice owns a dedicated, physically isolated MySQL database schema (`aerobook_auth_db` , `aerobook_user_db` , etc.). Direct cross-service database querying is strictly prohibited.
4. **Stateless JWT Security & Token Propagation**: Cryptographically signed HMAC-SHA256 tokens authenticated at the Gateway. User identity is mutated into downstream HTTP headers (`X-User-Id` , `X-User-Email` , `X-User-Role`), liberating backend microservices from duplicate database token lookups.
5. **Hybrid Synchronous & Asynchronous Communication**:
   - **Synchronous (OpenFeign / REST)**: Used where immediate responses are strictly required (e.g. validating seat availability before confirming a reservation).
   - **Asynchronous (RabbitMQ Event Bus)**: Used for decoupled background tasks (e.g. `BOOKING_CREATED` events triggering inventory bookkeeping without blocking passenger responses).

2. Microservice Boundaries & Responsibilities

Service Identifier	Logical Name	Port	Database Schema	Core Business Boundary
`service-discovery`	Eureka Registry	8761	*None (In-Memory)*	Central service registry, peer discovery, and liveness monitoring
`config-server`	Spring Cloud Config	8080	*None (Local/Git)*	Centralized configuration repository for distributed service profiles
`api-gateway`	Cloud Gateway	8083	*None (Stateless)*	Reactive ingress reverse proxy, route routing, JWT filter, CORS, correlation tracing
`auth-service`	Authentication Service	8082	`aerobook_auth_db`	Customer registration, BCrypt password hashing, JWT lifecycle, OTP codes
`user-service`	User Profile Service	8084	`aerobook_user_db`	Customer demographic records, user directory, staff incident reporting tickets
`flight-service`	Flight Schedule Service	8087	`aerobook_flight_db`	Commercial flight schedules, route search, operational status dispatch, fleet roster
`fare-service`	Dynamic Fare Service	8089	`aerobook_fare_db`	Tiered cabin pricing, GST tax computation, promotional coupon discounts
`booking-service`	Reservation Service	8088	`aerobook_booking_db`	Flight reservations, PNR generation, passenger manifests, RabbitMQ event dispatch
`check-in-service`	Airport Check-In Service	8090	`aerobook_checkin_db`	Passenger airport check-in, seat assignment, boarding pass generation & gate scanning
`notification-service`	Alert & Document Service	8091	`aerobook_notification_db`	In-app alerts, Gmail SMTP dispatch, OpenPDF airline E-Ticket & Boarding Pass PDFs
`payment-service`	Razorpay Payment Service	8092	`aerobook_payment_db`	Razorpay order creation, HMAC-SHA256 signature verification, webhook processing, refunds

3. Ingress Routing & Filter Pipeline

The API Gateway is built on **Spring Cloud Gateway WebFlux**, utilizing non-blocking Netty sockets for high-concurrency throughput.

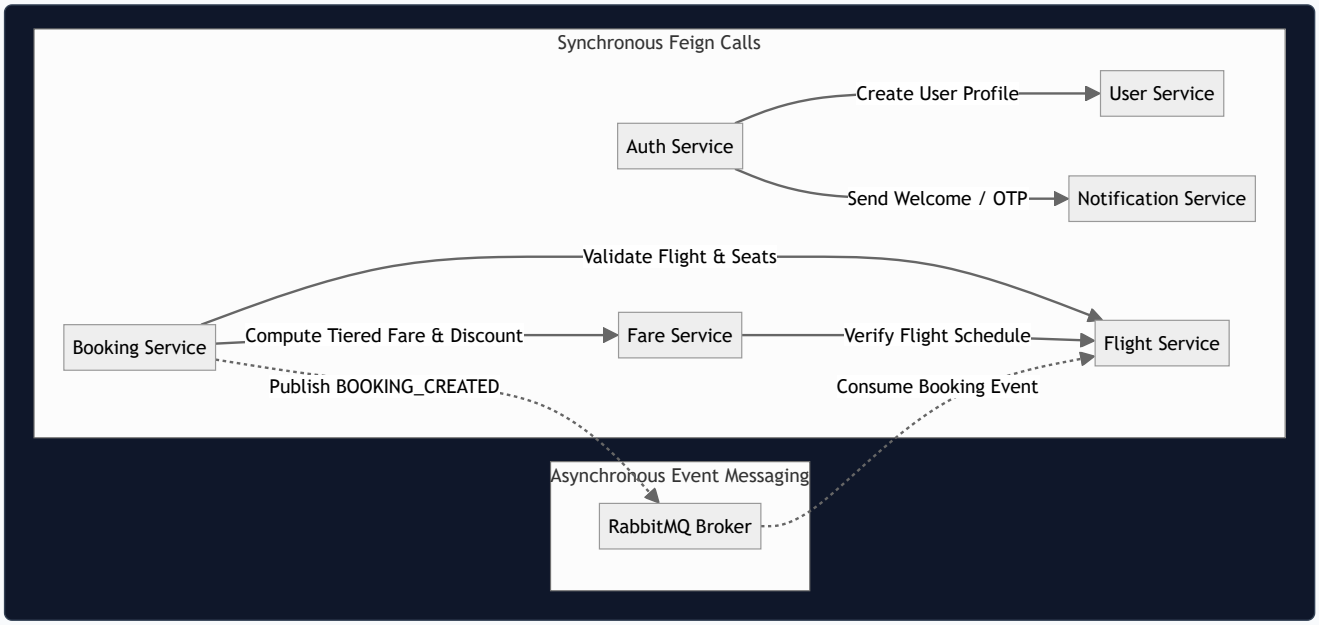


Gateway Cross-Cutting Filters:

1. **`CorrelationIdGlobalFilter`**: Generates or propagates an `X-Correlation-Id` across all hops to ensure distributed trace visibility in logs.
2. **`CorsDedupeGlobalFilter`**: Eliminates duplicate `Access-Control-Allow-Origin` headers that can occur when both the Gateway and embedded Tomcat instances attach CORS headers.
3. **`AuthenticationFilter`**: Verifies cryptographic HMAC-SHA256 signatures of JWT tokens, enforces token expiration limits, and injects user identity into downstream headers.

---

4. Inter-Service Communication Matrix



4.1 Synchronous Invocations (OpenFeign)

- **`Auth Service → User Service`**: During `/api/auth/register`, `UserClient` invokes `POST /api/v1/users` to synchronize customer demographics into `aerobook_user_db`.
- **`Auth Service → Notification Service`**: Invokes `POST /api/notifications/email/verify` to dispatch 6-digit OTP codes via SMTP.
- **`Booking Service → Flight Service`**: `FlightServiceClient` invokes `GET /api/flights/{id}` to verify schedule status and check that `available_seats > 0`.
- **`Booking Service → Fare Service`**: `FareServiceClient` invokes `GET /api/fares/flight/{flightId}` to calculate exact cabin pricing and apply validated coupons.
- **`Fare Service → Flight Service`**: `FlightServiceClient` verifies flight schedule existence before permitting administrators to create or update tiered pricing models.

4.2 Asynchronous Event Flow (RabbitMQ)

- **Producer**: `BookingEventPublisher` in `booking-service` publishes message payload `BOOKING_CREATED:<pnr>` to direct exchange `booking-exchange` with routing key `booking-created`.
- **Consumer**: `BookingEventConsumer` in `flight-service` listens on queue `booking-created` to asynchronously update internal flight metrics and booking counters.
- **Resilience**: Configured with fallback mechanisms so that if RabbitMQ is unavailable in development, booking operations complete without throwing exceptions.

---

5. Architectural Trade-offs & Resilience

Architecture Decision	Trade-Off Accepted	Mitigating Design Implementation
Database per Service	Cannot execute SQL `JOIN` across booking and flight tables	Implemented OpenFeign clients and consolidated response DTOs
Reactive Gateway Engine	Netty reactive pipelines require non-blocking code and distinct mental models	Strict separation between reactive Gateway layer and traditional MVC servlet services
Stateless JWT Authentication	Revoking compromised tokens immediately requires token blacklisting	Short-lived access tokens (24 hours) coupled with refresh token rotation stored in `aerobook_auth_db`
Synchronous Booking Validation	Booking latency depends on Flight and Fare service response times	Fast in-memory database queries with indexed primary keys; sub-50ms execution times
Zero Mock Frontend	Local development requires active backend services	Orchestrator scripts (`run-services.ps1` , `start-all.bat`) allow 1-click startup of all 11 services

CHAPTER 4

04. Repository & Project Structure

1. Monorepo Architecture Overview

AeroBook is organized as a structured multi-module monorepo containing 11 backend microservices, a standalone Angular frontend application, operational automation scripts, centralized database migration scripts, and continuous quality inspection pipelines.

```
d:\aerobook/
├── .mvn/                              # Maven wrapper binary and properties
├── docs/                              # Complete comprehensive technical documentation
│   ├── 05-microservices/             # Dedicated deep-dive microservice specifications
│   └── Complete_Project_Documentation.pdf # Consolidated master PDF publication
├── scripts/                           # Orchestration, database seeds, and verification scripts
├── service-discovery/                 # Netflix Eureka Service Registry (Port 8761)
├── config-server/                     # Spring Cloud Config Server (Port 8080)
├── api-gateway/                       # Spring Cloud Gateway WebFlux Reverse Proxy (Port 8083)
├── auth-service/                      # Identity, JWT & Security Microservice (Port 8082)
├── user-service/                      # User Profiles & Incident Microservice (Port 8084)
├── flight-service/                    # Flight Catalog & Fleet Microservice (Port 8087)
├── fare-service/                      # Dynamic Fares & Coupons Microservice (Port 8089)
├── booking-service/                   # Reservations & PNR Microservice (Port 8088)
├── check-in-service/                  # Check-In & Boarding Pass Microservice (Port 8090)
├── notification-service/              # Alerts, SMTP & PDF Generator Microservice (Port 8091)
├── payment-service/                   # Razorpay Orders & Webhook Microservice (Port 8092)
├── frontend/                          # Angular 18 Single Page Application (Port 4200)
├── .env.example                       # Environment configuration template
├── .env.sonarqube.example             # SonarQube & PostgreSQL environment template
├── docker-compose.sonarqube.yml       # Containerized SonarQube & PostgreSQL stack
├── pom.xml                            # Root Maven aggregator POM for all 11 microservices
├── mvnw / mvnw.cmd                    # Cross-platform Maven wrapper executables
├── run-sonar-analysis.ps1 / .bat      # Master SonarQube code quality pipeline
└── start-all.bat / stop-all.bat       # 1-Click developer lifecycle scripts
```

2. Root Files & Operational Scripts

File Name	Purpose & Function
`pom.xml`	Root Maven aggregator orchestrating multi-module builds, tests, and static analysis across all 11 microservices
`mvnw` / `mvnw.cmd`	Maven 3.9.16 Wrapper enabling reproducible builds without requiring globally installed Maven
`.env.example`	Canonical environment variable reference detailing MySQL databases, JWT secret, Razorpay credentials, and service ports
`docker-compose.sonarqube.yml`	Production-ready Compose definition running SonarQube 10.5 Community + PostgreSQL 15 with persistent named volumes
`run-sonar-analysis.ps1`	PowerShell orchestrator checking server availability, compiling, running tests, validating JaCoCo/LCOV, and triggering SonarQube
`run-sonar-analysis.bat`	Native Windows Command Prompt wrapper for `run-sonar-analysis.ps1`
`start-all.bat` / `stop-all.bat`	Quick lifecycle batch triggers delegating to `scripts/run-services.ps1`
`scripts/run-services.ps1`	Advanced service orchestrator implementing tiered startup ordering (Eureka first, then business services), health polling, and process management
`scripts/seed.sql`	Master SQL migration script creating all 8 databases and populating 15-30 realistic aviation records per domain

3. Backend Microservice Internal Architecture

Every Spring Boot microservice adheres to a clean, standardized layered architecture:

```
<service-directory>/
├── pom.xml                          # Service-specific build manifest, plugins, dependencies
├── mvnw / mvnw.cmd                  # Embedded service-level Maven wrapper
├── src/
│   ├── main/
│   │   ├── java/com/aerobook/<service>/
│   │   │   ├── <Service>Application.java # Spring Boot entry point & bootstrap configuration
│   │   │   ├── client/              # OpenFeign declarative REST client interfaces
│   │   │   ├── config/              # Security, WebMvc, RabbitMQ, and CORS configuration classes
│   │   │   ├── controller/          # Spring MVC REST Controllers exposing HTTP endpoints
│   │   │   ├── dto/                 # Request payloads, response bodies, and error records
│   │   │   ├── entity/              # JPA Entities mapped to MySQL database tables
│   │   │   ├── exception/           # Domain-specific exceptions and GlobalExceptionHandler
│   │   │   ├── messaging/           # RabbitMQ event producers and consumers
│   │   │   ├── repository/          # Spring Data JPA Repository interfaces
│   │   │   ├── service/             # Core business logic interfaces
│   │   │   │   └── impl/            # Business service implementations
│   │   │   └── util/                # Helper utilities, validators, and token parsers
│   │   └── resources/
│   │       ├── application.properties   # Microservice port, MySQL datasource, and Eureka settings
│   │       └── logback-spring.xml       # Structured logging layout and console patterns
│   └── test/
│       └── java/com/aerobook/<service>/ # Unit and integration test suites (JUnit 5 & Mockito)
└── target/
    ├── classes/                     # Compiled Java bytecode
    └── site/jacoco/jacoco.xml       # Automated JaCoCo XML code coverage report
```

Key Service Package Directory Mappings:

- **`client/`**: Declarative `@FeignClient` interfaces specifying logical Eureka targets (`name = "FLIGHT-SERVICE"`).
- **`config/`**: Contains Spring configuration beans, security filters, and RabbitMQ queue/exchange bindings.
- **`controller/`**: Handles HTTP serialization, path routing, and parameter validation (`@Valid`).
- **`dto/`**: Strongly typed data transfer objects decoupling internal persistence models from API responses.
- **`entity/`**: Database records decorated with Jakarta Persistence annotations (`@Entity` , `@Table` , `@Id`).
- **`repository/`**: Inherits `JpaRepository<T, ID>` with custom query derivations and JPQL definitions.
- **`service/impl/`**: Pure business rules, validation constraints, and transactional boundaries (`@Transactional`).

---

4. Frontend Application Structure

The Angular frontend (`frontend/`) utilizes Angular 18's Standalone Component architecture, eliminating legacy `NgModule` boilerplate and structuring code by functional concern:

```
frontend/
├── angular.json                     # Angular CLI workspace build and test configurations
├── package.json                     # npm dependencies, scripts, and build tasks
├── tsconfig.json / tsconfig.app.json    # TypeScript compiler configuration
├── tsconfig.spec.json               # TypeScript configuration for Jasmine unit tests
├── karma.conf.js                    # Karma test runner config with ChromeHeadlessCI & LCOV
├── sonar-project.properties         # SonarQube scanner properties for TypeScript analysis
├── public/                          # Static public web assets (favicons, manifest)
└── src/
    ├── index.html                   # HTML shell with zero-flash theme bootstrap script
    ├── main.ts                      # Angular platform bootstrap entry point
    ├── styles.css                   # Global Tailwind CSS and dual-theme CSS variable tokens
    └── app/
        ├── app.component.ts         # Root application shell rendering navbar & router outlet
        ├── app.routes.ts            # Client-side route declarations and guard attachments
        ├── app.config.ts            # Application providers (HttpClient, router, animations)
        ├── core/                    # Singleton core infrastructure (guards, interceptors, API)
        │   ├── api/                 # Strongly typed API client layer (16 domain clients)
        │   │   ├── api.service.ts   # Central HTTP client handling base URL & error mapping
        │   │   ├── auth.api.ts      # Login, register, refresh token, OTP endpoints
        │   │   ├── flights.api.ts   # Flight search, details, and status updates
        │   │   ├── bookings.api.ts  # Reservation creation, PNR search, cancel
        │   │   ├── payments.api.ts  # Razorpay order creation and verification
        │   │   └── ...              # Check-in, notifications, fares, coupons, users APIs
        │   ├── guards/              # Route guards (customerGuard, staffGuard, adminGuard)
        │   ├── interceptors/        # authInterceptor attaching Bearer token & correlation IDs
        │   ├── models/              # TypeScript interface definitions (aerobook.models.ts)
        │   ├── services/            # State services (AuthService, ToastService, BookingService)
        │   └── theme/               # ThemeService managing light/dark mode and localStorage
        ├── features/                # Role-specific business views and dashboards
        │   ├── home/                # Public landing page (home.component.ts)
        │   ├── customer/            # Passenger dashboard (customer-dashboard.component.ts)
        │   ├── staff/               # Airport ground staff portal (staff-dashboard.component.ts)
        │   └── admin/               # Airline admin control center (admin-dashboard.component.ts)
        └── shared/                  # Reusable visual components and design primitives
            ├── components/          # Complex shared widgets (navbar, login-modal, register-modal)
            └── ui/                  # Atomic design system components (ab-button, ab-card, etc.)
```

Design System Primitives (`shared/ui/`):

- **`ab-button.component.ts`**: Standardized buttons with variant styles (primary, outline, ghost, danger).

- `ab-card.component.ts` : Glassmorphic panels with light/dark adaptive background and borders.
- `ab-input.component.ts` : Text, password, and number inputs with floating labels and error states.
- `ab-modal.component.ts` : Accessible dialog overlays with backdrop blur and escape key handling.
- `ab-status-pill.component.ts` : Colored status badges for flight and booking states.
- `ab-ticket.component.ts` : Specialized airline ticket presentation widget.

CHAPTER 5.1

Microservice: Service Discovery (Eureka Server)

1. Executive Summary & Purpose

The **Service Discovery** microservice serves as the central registration directory and heart of the AeroBook distributed platform. Powered by **Netflix Eureka Server**, it dynamically registers, monitors, and tracks the health and network coordinates of all microservice instances across the ecosystem.

- **Logical Service Name**: `service-discovery`
- **Eureka Application ID**: `SERVICE-DISCOVERY`
- **Default Port**: `8761`
- **Dashboard URL**: `http://localhost:8761`
- **Database Schema**: None (In-memory replicated state)

---

2. Service Boundaries & Responsibilities

- **Central Registry**: Maintains an active registry of all running microservice instances, IPs, and ports.
- **Heartbeat & Liveness Monitoring**: Listens for periodic heartbeat renewal requests (default: every 30s) from client services; evicts expired instances.
- **Dynamic Routing Enablement**: Enables the API Gateway and OpenFeign clients to route requests using logical identifiers (`lb://AUTH-SERVICE` , `lb://FLIGHT-SERVICE`) rather than hardcoded IP addresses.
- **Self-Preservation Mode**: Prevents mass registry eviction during temporary network partitions.

---

3. Technology Stack & Dependencies

- **Language**: Java 17
- **Framework**: Spring Boot 4.1.1
- **Cloud Orchestration**: Spring Cloud 2025.1.3 (`spring-cloud-starter-netflix-eureka-server`)
- **Web Runtime**: Spring Boot Starter WebMVC with embedded Apache Tomcat
- **Operational Metrics**: Spring Boot Starter Actuator
- **Build Tool**: Apache Maven 3.9.16 via Maven Wrapper (`mvnw.cmd`)
- **Quality & Coverage**: `jacoco-maven-plugin:0.8.12` , `sonar-maven-plugin:5.0.0.4389`

---

4. Bootstrap & Entry Point

Entry Point: **ServiceDiscoveryApplication.java**

```java
package com.aerobook.discovery;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class ServiceDiscoveryApplication {
    public static void main(String[] args) {
        SpringApplication.run(ServiceDiscoveryApplication.class, args);
    }
}
```

The `@EnableEurekaServer` annotation initializes the Eureka controller, replication channels, and in-memory peer awareness registry.

---

5. Configuration & Environment Variables

Configuration: **application.properties**

```
spring.application.name=service-discovery
server.port=8761

# Standalone local deployment settings
eureka.instance.hostname=localhost
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.client.service-url.defaultZone=http://${eureka.instance.hostname}:${server.port}/eureka/

# Eviction and heartbeat timing
eureka.server.enable-self-preservation=false
eureka.server.eviction-interval-timer-in-ms=5000

# Actuator monitoring
management.endpoints.web.exposure.include=health,info
```

Property	Default	Purpose
`server.port`	`8761`	Eureka HTTP listener port
`eureka.client.register-with-eureka`	`false`	Prevents the server from registering with itself
`eureka.client.fetch-registry`	`false`	Disables client registry caching for standalone server
`eureka.server.enable-self-preservation`	`false`	Allows immediate eviction during development restarts

6. Endpoints & API Reference

Method	Endpoint	Access	Purpose
`GET`	`/`	Public	HTML Eureka Web Dashboard displaying active instances
`GET`	`/eureka/apps`	Internal	XML/JSON registry catalog of all registered applications
`GET`	`/eureka/apps/{appId}`	Internal	Specific microservice instance registry details
`POST`	`/eureka/apps/{appId}`	Internal	Client instance registration
`PUT`	`/eureka/apps/{appId}/{instanceId}`	Internal	Client 30-second heartbeat renewal
`DELETE`	`/eureka/apps/{appId}/{instanceId}`	Internal	Client graceful deregistration on shutdown
`GET`	`/actuator/health`	Public	Spring Boot Actuator liveness and health probe

7. Testing & Build Instructions

Running Tests & JaCoCo Coverage:

```
cd d:\aerobook\service-discovery
.\mvnw.cmd test
```

- Tests run: `ServiceDiscoveryApplicationTests`
- Output JaCoCo report: `target/site/jacoco/jacoco.xml`

Starting the Service:

```
.\mvnw.cmd spring-boot:run
```

Upon startup, navigate to `http://localhost:8761` to view registered microservices.

CHAPTER 5.2

Microservice: Config Server (Spring Cloud Config)

1. Executive Summary & Purpose

The **Config Server** provides optional centralized external configuration management for microservices across development, testing, and production environments. Powered by **Spring Cloud Config Server**, it allows configuration properties to be version-controlled and updated without rebuilding or redeploying microservice binaries.

- **Logical Service Name**: `config-server`
- **Default Port**: `8080`
- **Health Probe**: `http://localhost:8080/actuator/health`
- **Database Schema**: None (File system or Git repository storage)

---

2. Service Boundaries & Responsibilities

- **Centralized Environment Profiles**: Serves environment-specific property files (`application-dev.properties` , `application-prod.properties`) to downstream services.
- **Dynamic Configuration Refresh**: Integrates with Spring Cloud Bus or actuator refresh endpoints to hot-reload properties.
- **Secret Encryption & Decryption**: Supports symmetric and asymmetric decryption of sensitive property values.

---

3. Technology Stack & Dependencies

- **Language**: Java 17
- **Framework**: Spring Boot 4.1.1
- **Cloud Orchestration**: Spring Cloud 2025.1.3 (`spring-cloud-config-server`)
- **Web Runtime**: Spring Boot Starter WebMVC
- **Operational Metrics**: Spring Boot Starter Actuator
- **Build Tool**: Apache Maven 3.9.16 via Maven Wrapper (`mvnw.cmd`)
- **Coverage & Static Analysis**: `jacoco-maven-plugin:0.8.12` , `sonar-maven-plugin:5.0.0.4389`

---

4. Bootstrap & Entry Point

Entry Point: **`ConfigServerApplication.java`**

```
package com.aerobook.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

The `@EnableConfigServer` annotation activates the property resolution endpoints and configuration loaders.

---

5. Configuration & Environment Variables

Configuration: **`application.properties`**

```
spring.application.name=config-server
server.port=8080

# Profile storage location (native filesystem or git)
spring.profiles.active=native
spring.cloud.config.server.native.search-locations=classpath:/config

# Actuator monitoring
management.endpoints.web.exposure.include=health,info,env
```

Property	Default	Purpose
`server.port`	`8080`	Port on which the config server listens

Property	Default	Purpose
`spring.profiles.active`	`native`	Storage backend mode (native filesystem or git)
`management.endpoints.web.exposure.include`	`health,info,env`	Exposed actuator endpoints

6. Endpoints & API Reference

Method	Endpoint	Access	Purpose
`GET`	`/{application}/{profile}`	Client Services	Fetches active properties for a specific service and environment profile
`GET`	`/{application}/{profile}/{label}`	Client Services	Fetches properties tied to a specific branch or version label
`GET`	`/{application}-{profile}.properties`	Client Services	Formatted Java properties output for legacy consumers
`GET`	`/{application}-{profile}.json`	Client Services	JSON formatted property representation
`GET`	`/actuator/health`	Public	Liveness probe indicating server status

7. Testing & Build Instructions

Running Tests & JaCoCo Coverage:

```
cd d:\aerobook\config-server
.\mvnw.cmd test
```

- Test Suite: `ConfigServerApplicationTests`
- Output JaCoCo report: `target/site/jacoco/jacoco.xml`

Starting the Service:

```
.\mvnw.cmd spring-boot:run
```

Downstream microservices can import configurations via `spring.config.import=optional:configserver:http://localhost:8080`.

CHAPTER 5.3

Microservice: API Gateway (Spring Cloud Gateway WebFlux)

1. Executive Summary & Purpose

The **API Gateway** microservice is the intelligent ingress reverse proxy and central traffic controller of the AeroBook platform. Built on **Spring Cloud Gateway 2025** using the non-blocking **Spring WebFlux (Netty)** reactive engine, it provides unified API routing, dynamic client-side load balancing via Eureka, cryptographic JWT authentication, role-based authorization, CORS termination, distributed correlation tracing, and platform diagnostics.

- **Logical Service Name**: `api-gateway`
- **Default Port**: `8083`
- **Health Endpoint**: `http://localhost:8083/api/gateway/health`
- **Routing Engine**: Spring WebFlux Reactive (Netty)
- **Database Schema**: None (Stateless ingress proxy)

---

2. Service Boundaries & Key Responsibilities

1. **Unified API Ingress**: Provides a single host and port (`localhost:8083`) for all frontend communication.
2. **Dynamic Service Resolution**: Resolves target microservices via Netflix Eureka using `lb://<SERVICE_ID>`.
3. **Cryptographic JWT Validation**: Intercepts secured routes, verifies HMAC-SHA256 signatures, and checks token expiration.
4. **Downstream Identity Mutation**: Injects validated user identity headers (`X-User-Id` , `X-User-Email` , `X-User-Role`) before forwarding requests to internal services.
5. **Distributed Correlation Tracing**: Injects unique `X-Correlation-Id` tracking identifiers on all incoming requests.
6. **Global CORS Deduplication**: Deduplicates `Access-Control-Allow-Origin` headers between gateway and downstream services.
7. **Platform Architecture Hub**: Exposes `/api/gateway/services` and `/api/gateway/rbac-matrix` diagnostic endpoints.

---

3. Technology Stack & Dependencies

- **Language**: Java 17
- **Framework**: Spring Boot 3.5.5
- **Cloud Orchestration**: Spring Cloud 2025.0.0
- **Reactive Engine**: `spring-cloud-starter-gateway-server-webflux` (Project Reactor & Netty)
- **Service Discovery**: `spring-cloud-starter-netflix-eureka-client`
- **Security & Tokens**: `io.jsonwebtoken:jjwt-api:0.12.7` , `jjwt-impl:0.12.7` , `jjwt-jackson:0.12.7`
- **Observability**: `spring-boot-starter-actuator`
- **Testing**: JUnit Jupiter, `io.projectreactor:reactor-test`
- **Quality & Coverage**: `jacoco-maven-plugin:0.8.12` , `sonar-maven-plugin:5.0.0.4389`

---

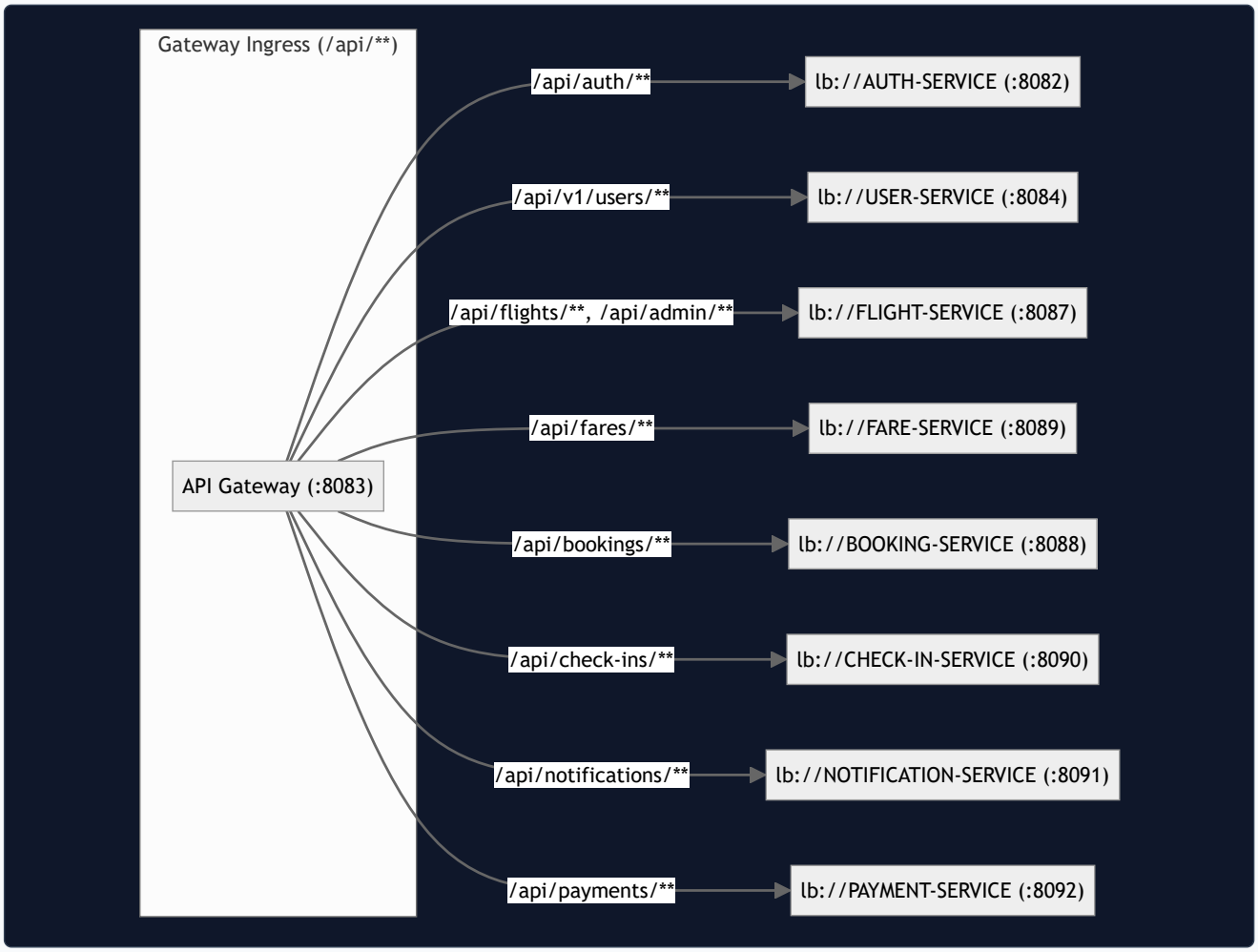
4. Internal Architecture & Source Structure

```
api-gateway/src/main/java/com/aerobook/apigateway/
├── ApiGatewayApplication.java           # Spring Boot reactive bootstrap
├── config/
│   └── GatewayConfig.java               # Declarative RouteLocator route table & filter chain
├── controller/
│   └── GatewayOverviewController.java   # Diagnostics, services directory, and RBAC matrix
├── dto/
│   └── ErrorResponse.java               # Structured JSON gateway error payload
├── exception/
│   └── GlobalExceptionHandler.java      # Reactive ErrorWebExceptionHandler
├── filter/
│   ├── AuthenticationFilter.java        # Custom GatewayFilterFactory for JWT validation
│   ├── CorrelationIdGlobalFilter.java   # Global filter adding X-Correlation-Id
│   └── CorsDedupeGlobalFilter.java      # Global filter cleaning duplicate CORS headers
├── util/
│   └── JwtUtil.java                     # HMAC-SHA token parser and claims extractor
└── validator/
    └── RouteValidator.java              # Predicate classifying open vs secured routes
```

---

5. Route Table & Forwarding Configuration

Configured in `GatewayConfig.java` :



Route ID	Ingress Path Predicate	Filter Chain	Target Destination
`auth-service`	`/api/auth/**`	`AuthenticationFilter`	`lb://AUTH-SERVICE`
`user-service`	`/api/v1/users`, `/api/v1/users/**`	`AuthenticationFilter`	`lb://USER-SERVICE`
`flight-service`	`/api/flights/**`, `/api/admin/flights/**`, `/api/admin/aircrafts/**`	`AuthenticationFilter`	`lb://FLIGHT-SERVICE`
`fare-service`	`/api/fares`, `/api/fares/**`	`AuthenticationFilter`	`lb://FARE-SERVICE`
`booking-service`	`/api/bookings`, `/api/bookings/**`	`AuthenticationFilter`	`lb://BOOKING-SERVICE`
`check-in-service`	`/api/check-ins`, `/api/check-ins/**`	`AuthenticationFilter`	`lb://CHECK-IN-SERVICE`
`notification-service`	`/api/notifications`, `/api/notifications/**`	`AuthenticationFilter`	`lb://NOTIFICATION-SERVICE`
`payment-service`	`/api/payments`, `/api/payments/**`	`AuthenticationFilter`	`lb://PAYMENT-SERVICE`

6. Security & Authentication Filter Flow

The `AuthenticationFilter.java` executes on every route:

1. **Whitelisting Inspection**: Queries `RouteValidator.java`. If path is in `openApiEndpoints` or ends with `/health`, the request passes through unhindered.
2. **Bearer Token Extraction**: Inspects the `Authorization` header. If missing or malformed, returns `HTTP 401 Unauthorized`.
3. **Cryptographic Signature Verification**: Validates the JWT token using `jwt.secret`.
4. **Claims Extraction & Header Mutation**:

```
ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
    .header("X-User-Id", String.valueOf(claims.get("userId")))
    .header("X-User-Email", claims.getSubject())
    .header("X-User-Role", String.valueOf(claims.get("role")))
    .build();
```

5. **Downstream Execution**: Forwards the enriched request to the downstream target microservice.

---

7. Platform Diagnostics Endpoints (`GatewayOverviewController.java`)

Method	Endpoint	Access	Response Content
`GET`	`/api/gateway/health`	Public	Status (`UP`), port (`8083`), routing engine, Eureka registration state
`GET`	`/api/gateway/services`	Public	Metadata, ports, databases, and responsibilities for all services
`GET`	`/api/gateway/endpoints`	Public	Exhaustive list of platform endpoints, HTTP methods, and required roles
`GET`	`/api/gateway/rbac-matrix`	Public	Detailed Role-Based Access Control matrix across all user personas
`GET`	`/api/gateway/presentation-script`	Public	Step-by-step evaluator testing flow for live demonstrations

---

8. Testing & Validation

Running Unit & StepVerifier Tests:

```
cd d:\aerobook\api-gateway
.\mvnw.cmd test
```

- Tests run: 9 passed (`ApiGatewayApplicationTests` , `GatewayOverviewControllerTest` , `RouteValidatorTest`)
- JaCoCo Report: `target/site/jacoco/jacoco.xml` (77 KB)

Starting the Gateway:

```
.\mvnw.cmd spring-boot:run
```

Once booted, all incoming frontend API traffic passes through `http://localhost:8083` .

CHAPTER 5.4

Microservice: Auth Service (Authentication & Identity)

1. Executive Summary & Purpose

The **Auth Service** is the central identity provider and security authority for the AeroBook platform. It manages user credentials, performs one-way cryptographic password hashing (BCrypt), issues cryptographically signed JSON Web Tokens (JWT), manages refresh token lifecycles, coordinates email OTP verification, and enforces Role-Based Access Control (RBAC).

- **Logical Service Name**: `auth-service`
- **Port**: `8082`
- **Database Schema**: `aerobook_auth_db` (MySQL 8.x)
- **Primary Endpoint**: `/api/auth`
- **Health Probe**: `http://localhost:8082/api/auth/health`

---

2. Service Boundaries & Responsibilities

1. **Customer Registration**: Validates new user accounts, stores BCrypt-hashed passwords, and synchronizes profile data to `user-service` via OpenFeign.
2. **Privileged Account Provisioning**: Protects administrative (`ROLE_ADMIN`) and ground operations (`ROLE_STAFF`) onboarding via secret setup keys (`X-Admin-Setup-Key` , `X-Staff-Setup-Key`).
3. **Authentication & Token Issuance**: Verifies credentials and generates paired JWT tokens:
   - **Access Token**: Short-lived (24 hours), carrying user claims (`userId` , `email` , `role`).
   - **Refresh Token**: Long-lived (7 days), stored in `aerobook_auth_db` for rotating access tokens.
4. **Email OTP Verification**: Generates and verifies 6-digit one-time passcodes for email confirmation and self-service password resets.
5. **Administrative Access Auditing**: Exposes credential listings and role elevation/demotion endpoints.

---

3. Technology Stack & Dependencies

- **Language**: Java 17
- **Framework**: Spring Boot 4.1.1
- **Cloud Orchestration**: Spring Cloud 2025.1.3 (`spring-cloud-starter-netflix-eureka-client` , `spring-cloud-starter-openfeign`)
- **Persistence**: Spring Data JPA / Hibernate 7.4.5, MySQL Connector/J
- **Security & Cryptography**: Spring Security Crypto (`BCryptPasswordEncoder`), `io.jsonwebtoken:jjwt:0.12.7`
- **Build Tool**: Maven 3.9.16 via `mvnw.cmd`
- **Coverage & Static Analysis**: `jacoco-maven-plugin:0.8.12` , `sonar-maven-plugin:5.0.0.4389`

---

4. Database Design & Entity Model

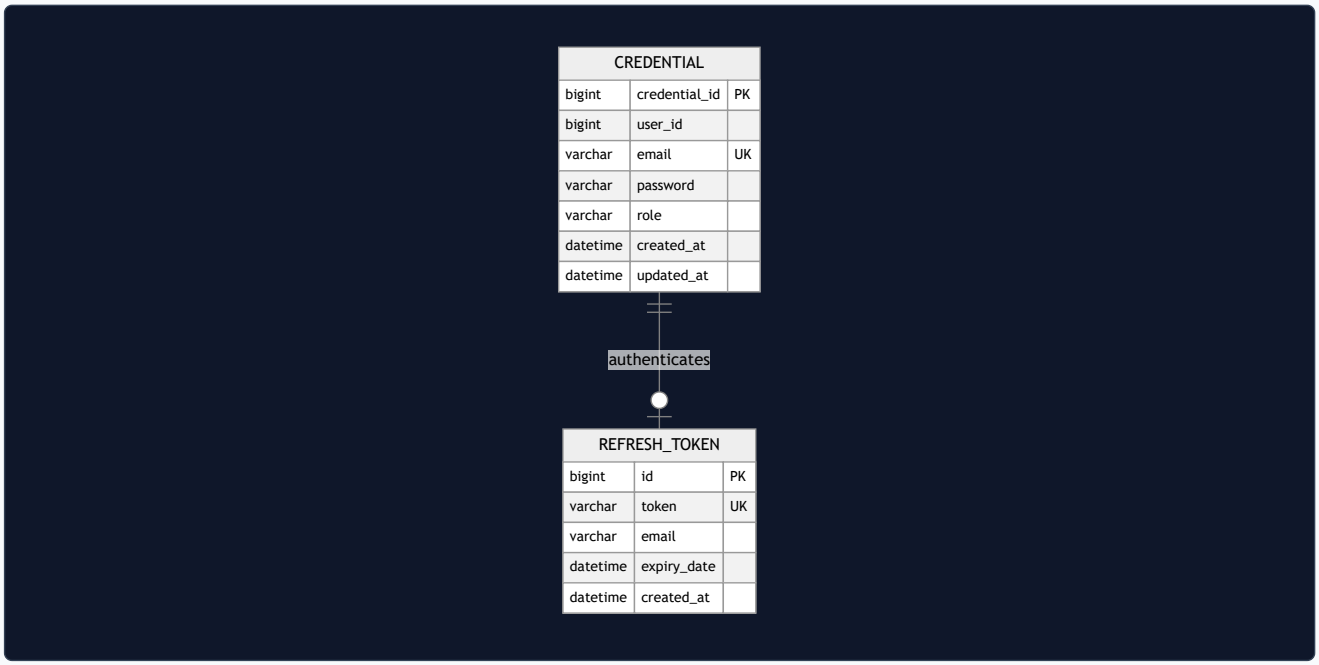


Table: **`aerobook_auth_db.credentials`**

- `credential_id` (BIGINT, PK, Auto Increment)
- `user_id` (BIGINT, Not Null) - Matches `user_id` in `aerobook_user_db.users`
- `email` (VARCHAR(100), Unique, Not Null)
- `password` (VARCHAR(255), Not Null) - Stored as standard BCrypt hash (`$2a$10$...`)
- `role` (VARCHAR(20), Not Null) - Values: `ROLE_USER` , `ROLE_STAFF` , `ROLE_ADMIN`
- `created_at` / `updated_at` (DATETIME)

---

5. Endpoints & API Reference

Method	Endpoint	Auth Required	Purpose
`GET`	`/api/auth/health`	Public	Operational health status probe
`POST`	`/api/auth/register`	Public	Customer account registration (`ROLE_USER`)
`POST`	`/api/auth/register-admin`	`X-Admin-Setup-Key`	Privileged admin registration (`ROLE_ADMIN`)
`POST`	`/api/auth/register-staff`	`X-Staff-Setup-Key`	Privileged staff registration (`ROLE_STAFF`)
`POST`	`/api/auth/login`	Public	Credential authentication returning JWT access & refresh tokens
`POST`	`/api/auth/refresh-token`	Public	Refresh token exchange for new access token
`POST`	`/api/auth/send-verification-code`	Public	Dispatches 6-digit email confirmation OTP
`POST`	`/api/auth/verify-code`	Public	Verifies 6-digit confirmation OTP
`POST`	`/api/auth/forgot-password`	Public	Generates password reset OTP code
`POST`	`/api/auth/reset-password`	Public	Sets new password upon OTP verification
`GET`	`/api/auth/credentials`	`ROLE_ADMIN`	Audits all registered system credentials
`PUT`	`/api/auth/credentials/{userId}/role`	`ROLE_ADMIN`	Modifies user authority (`ROLE_USER` ↔ `ROLE_ADMIN`)

Example Request: **`POST /api/auth/login`**

```
{
  "email": "asha.khan@example.com",
  "password": "Password123!"
}
```

Example Successful Response (HTTP 200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhc2hhLmtoYW5AZXhhbXBsZS5jb20iLCJ1c2VySWQiOjEsInJvbGUiOiJST0xFX1VTRVIiLCJpYXQiOjE3MjkxNzAwMDAsImV4cCI6
  "refreshToken": "48b671e6-0e1a-4d2a-89a6-773a985f1c30",
  "tokenType": "Bearer",
  "expiresIn": 86400000,
  "userId": 1,
  "email": "asha.khan@example.com",
  "role": "ROLE_USER"
}
```


6. Inter-Service Communication (OpenFeign)

1. `UserClient` (`user-service`):

```
@FeignClient(name = "USER-SERVICE")
public interface UserClient {
    @PostMapping("/api/v1/users")
    UserResponse createUserProfile(@RequestBody UserRequest request);
}
```

   Invoked immediately upon customer registration to populate the personal profile in `aerobook_user_db` .

2. `NotificationClient` (`notification-service`):

```
@FeignClient(name = "NOTIFICATION-SERVICE")
public interface NotificationClient {
    @PostMapping("/api/notifications/email/verify")
    void sendVerificationEmail(@RequestBody EmailVerificationRequest request);
}
```

   Invoked to dispatch high-priority verification and reset code emails to the user's registered inbox.

7. Testing & Quality Verification

Running Test Suite:

```
cd d:\aerobook\auth-service
.\mvnw.cmd test
```

- Tests executed: 17 passed (`AuthServiceApplicationTests` , `AuthServiceImplTest`)
- JaCoCo Report: `target/site/jacoco/jacoco.xml` (119 KB)

CHAPTER 5.5

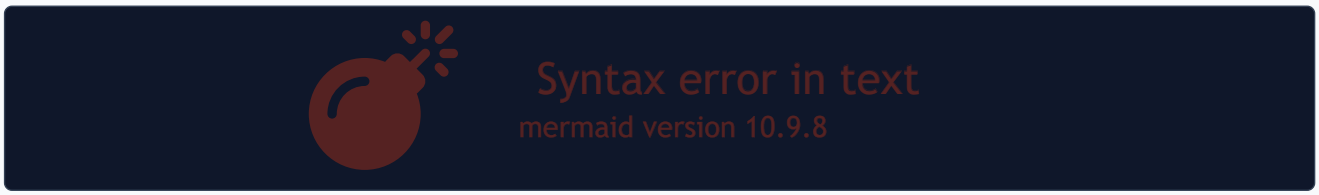
User Service Documentation

1. Executive Summary & Responsibility

The **User Service** (`user-service`) manages passenger profile records, customer demographic attributes, and airport operations staff incident reports. It operates as the central profile store for the AeroBook platform, decoupling customer identity metadata from authentication credentials (`auth-service`). It also provides an incident escalation and tracking system used by airline operations staff to flag passenger conduct, check-in issues, or security discrepancies.

- **Service Name**: `user-service`
- **Default Port**: `8084`
- **Application Context**: `/`
- **Base URI Path**: `/api/v1/users`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_user_db` (MySQL)
- **Service Discovery**: Registered in Eureka as `USER-SERVICE`

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_user_db.users`

Stores user profile information.

Column	Type	Constraints	Description
`user_id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Unique user profile identifier
`first_name`	`VARCHAR(100)`	NOT NULL	Passenger / user first name
`last_name`	`VARCHAR(100)`	NOT NULL	Passenger / user surname
`email`	`VARCHAR(150)`	NOT NULL, UNIQUE	User contact email
`phone_number`	`VARCHAR(20)`	NULL	Mobile contact number
`date_of_birth`	`DATE`	NULL	Date of birth for booking & TSA rules
`nationality`	`VARCHAR(50)`	NULL	Passenger nationality (e.g., Indian)
`role`	`VARCHAR(30)`	NOT NULL, DEFAULT `'ROLE_USER'`	Profile role (`ROLE_USER` , `ROLE_STAFF` , `ROLE_ADMIN`)
`created_at`	`DATETIME`	NOT NULL	Account creation timestamp
`updated_at`	`DATETIME`	NULL	Last modification timestamp

3.2 `aerobook_user_db.incident_reports`

Stores security and operational incident records submitted by staff or administrators regarding passengers.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Incident ticket ID
`user_id`	`BIGINT`	NOT NULL	Target passenger profile ID
`user_name`	`VARCHAR(100)`	NOT NULL	Full name of reported user
`user_email`	`VARCHAR(100)`	NOT NULL	Email address of reported user
`reported_by`	`VARCHAR(100)`	NOT NULL	Staff or admin username/email

Column	Type	Constraints	Description
`reason`	`VARCHAR(500)`	NOT NULL	Incident summary or violation
`severity`	`VARCHAR(20)`	NOT NULL, DEFAULT `'MEDIUM'`	`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`
`status`	`VARCHAR(20)`	NOT NULL, DEFAULT `'OPEN'`	`OPEN`, `INVESTIGATING`, `RESOLVED`, `CLOSED`
`notes`	`VARCHAR(1000)`	NULL	Internal operational logs and investigation notes
`created_at`	`DATETIME`	NOT NULL	Incident filing timestamp
`updated_at`	`DATETIME`	NULL	Last status update timestamp

4. REST API Endpoint Catalog

4.1 User Profiles (`UserController`)

Method	Path	Request Body	Response Body	Description	Access
`GET`	`/api/v1/users/health`	*None*	`String` ("User Service is Running")	Liveness probe	Public
`POST`	`/api/v1/users`	`UserRequest`	`UserResponse` (HTTP 201)	Create new user profile	Internal / Auth
`GET`	`/api/v1/users`	*None*	`List<UserResponse>`	Retrieve all user profiles	Admin / Staff
`GET`	`/api/v1/users/{id}`	*None*	`UserResponse`	Get profile by user ID	Authenticated
`GET`	`/api/v1/users/email/{email}`	*None*	`UserResponse`	Get profile by user email	Authenticated
`PUT`	`/api/v1/users/{id}`	`UserRequest`	`UserResponse`	Update user profile details	Owner / Admin
`DELETE`	`/api/v1/users/{id}`	*None*	`Void` (HTTP 204)	Delete user profile	Admin

4.2 Incident Reports (`IncidentController`)

Method	Path	Request Body	Response Body	Description	Access
`GET`	`/api/v1/users/incidents`	*None*	`List<IncidentReportResponse>`	List all filed incidents	Admin / Staff
`GET`	`/api/v1/users/incidents/{id}`	*None*	`IncidentReportResponse`	Get incident details by ticket ID	Admin / Staff
`POST`	`/api/v1/users/incidents`	`IncidentReportRequest`	`IncidentReportResponse` (HTTP 201)	Report passenger incident	Staff / Admin
`PUT`	`/api/v1/users/incidents/{id}/status`	Query: `status`	`IncidentReportResponse`	Update incident workflow status	Staff / Admin
`DELETE`	`/api/v1/users/incidents/{id}`	*None*	`Void` (HTTP 204)	Delete incident ticket	Admin
`GET`	`/api/v1/users/incidents/count/open`	*None*	`Long`	Count open incidents for staff dashboard	Staff / Admin

5. Key Dependencies & Configuration

- **Spring Cloud Config Client**: Fetches properties from `config-server:8080`.
- **Spring Cloud Eureka Client**: Registers with `service-discovery:8761`.
- **Spring Data JPA & Hibernate**: ORM mapping for MySQL `aerobook_user_db`.
- **ModelMapper / DTO Mapper**: Maps `UserEntity` and `IncidentReport` to clean response objects.
- **Actuator**: Health and metrics exposed under `/actuator`.

CHAPTER 5.6

Flight Service Documentation

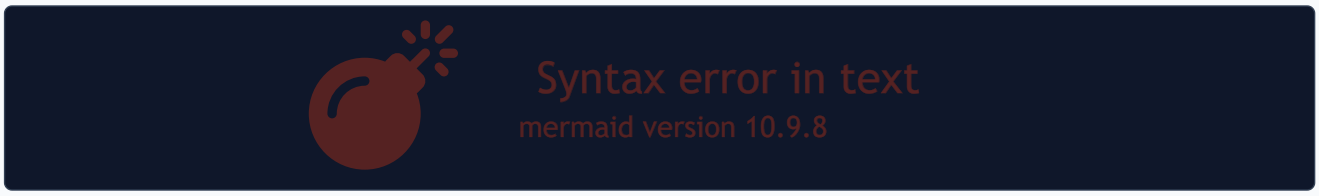
1. Executive Summary & Responsibility

The **Flight Service** (`flight-service`) manages airline flight schedules, aircraft inventory, seat capacity, real-time flight search, and flight lifecycle states. It serves as the authoritative source of truth for commercial airline routes, aircraft allocation, departure/arrival schedules, and available seat counts.

- **Service Name**: `flight-service`
- **Default Port**: `8087`
- **Application Context**: `/`
- **Base URI Paths**:
  - Public / Passenger Search: `/api/flights`
  - Admin Flight Operations: `/api/admin/flights`
  - Admin Aircraft Fleet: `/api/admin/aircrafts`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_flight_db` (MySQL)
- **Messaging**: RabbitMQ producer for `flight.queue` , consumer for `booking-created`
- **Service Discovery**: Registered in Eureka as `FLIGHT-SERVICE`

---

2. Architecture & Domain Model



---

3. Database Schema & Tables

3.1 `aerobook_flight_db.aircrafts`

Stores airline fleet records and seat capacity configurations.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Aircraft fleet entity ID
`aircraft_code`	`VARCHAR(50)`	NOT NULL, UNIQUE	Registration code (e.g. `VT-EXA` , `A320-NEO`)
`aircraft_name`	`VARCHAR(100)`	NOT NULL	Commercial model name (e.g. `Airbus A320neo`)
`manufacturer`	`VARCHAR(100)`	NOT NULL	Manufacturer (e.g. `Airbus` , `Boeing`)
`capacity`	`INT`	NOT NULL	Total passenger seating capacity (e.g. 180)
`status`	`VARCHAR(30)`	NOT NULL, DEFAULT `'ACTIVE'`	Fleet status (`ACTIVE` , `MAINTENANCE` , `RETIRED`)
`created_at`	`DATETIME`	NOT NULL	Aircraft induction timestamp
`updated_at`	`DATETIME`	NULL	Last modification timestamp

3.2 `aerobook_flight_db.flights`

Stores scheduled commercial flights, origin/destination city pairs, and real-time seat availability.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Flight schedule entity ID
`flight_number`	`VARCHAR(50)`	NOT NULL, UNIQUE	Commercial flight code (e.g. `AI-101` , `6E-204`)
`airline_name`	`VARCHAR(100)`	NOT NULL	Carrier name (e.g. `Air India` , `IndiGo`)
`source`	`VARCHAR(100)`	NOT NULL	Origin city / IATA code (e.g. `Delhi` , `DEL`)
`destination`	`VARCHAR(100)`	NOT NULL	Destination city / IATA code (e.g. `Mumbai` , `BOM`)

Column	Type	Constraints	Description
`departure_time`	`DATETIME`	NOT NULL	Scheduled local departure time
`arrival_time`	`DATETIME`	NOT NULL	Scheduled local arrival time
`total_seats`	`INT`	NOT NULL	Total passenger seats allocated
`available_seats`	`INT`	NOT NULL	Real-time remaining unreserved seats
`base_fare`	`DECIMAL(10,2)`	NOT NULL	Starting base price for economy ticket
`aircraft_id`	`BIGINT`	FOREIGN KEY (`aircrafts.id`)	Assigned aircraft from fleet
`status`	`VARCHAR(30)`	NOT NULL, DEFAULT `'SCHEDULED'`	Flight operational status

4. REST API Endpoint Catalog

4.1 Passenger & Public Search API (`FlightSearchController`)

Method	Path	Request Parameters	Response Body	Description	Access
`GET`	`/api/flights/health`	*None*	`String`	Health probe	Public
`GET`	`/api/flights/search`	`source` , `destination` , `date`	`List<FlightSearchResponse>`	Search flights matching route and departure date	Public
`GET`	`/api/flights`	*None*	`List<FlightResponse>`	Retrieve all scheduled flights	Public
`GET`	`/api/flights/{id}`	Path: `id`	`FlightResponse`	Get single flight details and seat inventory	Public

4.2 Fleet Management API (`AircraftController`)

Method	Path	Request Body	Response Body	Description	Access
`POST`	`/api/admin/aircrafts`	`CreateAircraftRequest`	`AircraftResponse` (HTTP 201)	Register a new aircraft to the fleet	Admin
`GET`	`/api/admin/aircrafts`	*None*	`List<AircraftResponse>`	List all aircraft in the airline fleet	Admin / Staff
`GET`	`/api/admin/aircrafts/{id}`	Path: `id`	`AircraftResponse`	Get aircraft details by fleet ID	Admin / Staff
`DELETE`	`/api/admin/aircrafts/{id}`	Path: `id`	`Void` (HTTP 204)	Delete or decommission an aircraft	Admin

4.3 Flight Operations Management API (`AdminFlightController`)

Method	Path	Request Body	Response Body	Description	Access
`POST`	`/api/admin/flights`	`CreateFlightRequest`	`FlightResponse` (HTTP 201)	Create and schedule a new flight	Admin
`PUT`	`/api/admin/flights/{id}`	`UpdateFlightRequest`	`FlightResponse`	Update schedule, status, or fare	Admin
`GET`	`/api/admin/flights`	*None*	`List<FlightResponse>`	Retrieve administrative flight roster	Admin / Staff
`GET`	`/api/admin/flights/{id}`	Path: `id`	`FlightResponse`	Retrieve detailed flight record	Admin / Staff
`DELETE`	`/api/admin/flights/{id}`	Path: `id`	`Void` (HTTP 204)	Cancel/delete scheduled flight	Admin

5. Asynchronous Messaging & Seat Synchronization

1. **Seat Deduction on Booking (`booking-created` queue)**:
   - When `booking-service` finalizes a reservation, it emits a `BookingCreatedEvent` .
   - `BookingEventConsumer` in `flight-service` intercepts the message, retrieves the corresponding `Flight` , and decrements `availableSeats` by the number of booked passengers in an atomic transaction.
2. **Flight Event Notification (`flight.queue`)**:
   - When flights are scheduled or rescheduled, `FlightEventProducer` publishes `FlightCreatedEvent` / `FlightUpdatedEvent` to notify downstream services.

CHAPTER 5.7

Fare Service Documentation

1. Executive Summary & Responsibility

The **Fare Service** (`fare-service`) is the dynamic pricing and promotional coupon engine of the AeroBook platform. It manages cabin class pricing tiers (Economy, Business, and First Class), dynamic discount calculations, tax percentages (standard Indian aviation GST), and promo coupon lifecycle management with validation constraints.

- **Service Name**: `fare-service`
- **Default Port**: `8089`
- **Application Context**: `/`
- **Base URI Paths**:
  - Flight Fares: `/api/fares`
  - Promotional Coupons: `/api/fares/coupons`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_fare_db` (MySQL)
- **Inter-Service Communication**: OpenFeign client (`FlightServiceClient`) to validate flight existence
- **Service Discovery**: Registered in Eureka as `FARE-SERVICE`

---

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_fare_db.fares`

Stores cabin pricing tiers and tax structures linked to flight schedules.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Fare record ID
`flight_id`	`BIGINT`	NOT NULL, UNIQUE	Associated Flight ID in `flight-service`
`economy_fare`	`DECIMAL(10,2)`	NOT NULL	Base Economy cabin ticket price
`business_fare`	`DECIMAL(10,2)`	NOT NULL	Base Business cabin ticket price
`first_class_fare`	`DECIMAL(10,2)`	NOT NULL	Base First Class cabin ticket price
`tax_percentage`	`DECIMAL(5,2)`	NOT NULL, DEFAULT `18.00`	Mandatory aviation tax / GST
`discount_percentage`	`DECIMAL(5,2)`	DEFAULT `0.00`	Base promotional discount
`effective_date`	`DATE`	NOT NULL	Fare schedule activation date
`active`	`TINYINT(1)`	NOT NULL, DEFAULT `1`	Active flag (`1` = True, `0` = False)

3.2 `aerobook_fare_db.coupons`

Stores promotional discount codes and redemption limits.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Coupon entity ID
`code`	`VARCHAR(50)`	NOT NULL, UNIQUE	Promotional code (e.g. `FLY500`, `AERO10`)
`description`	`VARCHAR(255)`	NOT NULL	Public promo description
`discount_percentage`	`DECIMAL(5,2)`	NOT NULL	Percentage markdown
`max_discount`	`DECIMAL(10,2)`	NOT NULL	Maximum discount cap in INR
`min_fare`	`DECIMAL(10,2)`	DEFAULT `0.00`	Minimum order fare required
`usage_limit`	`INT`	DEFAULT `1000`	Total redemptions allowed across platform
`usage_count`	`INT`	DEFAULT `0`	Running counter of redeemed coupons
`valid_from`	`DATE`	NULL	Promo start date
`valid_until`	`DATE`	NULL	Promo expiry date
`status`	`VARCHAR(20)`	NOT NULL, DEFAULT `'ACTIVE'`	Status (`ACTIVE`, `INACTIVE`, `EXPIRED`)
`created_at`	`DATETIME`	NULL	Creation timestamp
`updated_at`	`DATETIME`	NULL	Last modification timestamp

4. REST API Endpoint Catalog

4.1 Fare Pricing API (`FareController`)

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/fares/health`	*None*	`String`	Health check probe	Public
`POST`	`/api/fares`	`FareRequest`	`FareResponse` (HTTP 201)	Create cabin pricing for a flight	Admin
`GET`	`/api/fares/{id}`	Path: `id`	`FareResponse`	Get fare record by ID	Public / Services
`GET`	`/api/fares/flight/{flightId}`	Path: `flightId`	`FareResponse`	Get active fare structure for flight	Public / Services
`GET`	`/api/fares`	*None*	`List<FareResponse>`	List all flight fares	Public / Admin
`PUT`	`/api/fares/{id}`	`FareRequest`	`FareResponse`	Update cabin prices and taxes	Admin
`DELETE`	`/api/fares/{id}`	Path: `id`	`Void` (HTTP 204)	Delete fare record	Admin

4.2 Coupon Management API (`CouponController`)

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/fares/coupons`	*None*	`List<CouponResponse>`	List all available coupons	Public / Admin
`GET`	`/api/fares/coupons/{id}`	Path: `id`	`CouponResponse`	Get coupon by ID	Public / Admin
`POST`	`/api/fares/coupons`	`CouponRequest`	`CouponResponse` (HTTP 201)	Create new promotional coupon	Admin
`PUT`	`/api/fares/coupons/{id}`	`CouponRequest`	`CouponResponse`	Update coupon terms	Admin
`PUT`	`/api/fares/coupons/{id}/toggle`	Path: `id`	`CouponResponse`	Toggle coupon status (ACTIVE/INACTIVE)	Admin
`DELETE`	`/api/fares/coupons/{id}`	Path: `id`	`Void` (HTTP 204)	Delete promotional coupon	Admin
`POST`	`/api/fares/coupons/validate`	`CouponValidateRequest`	`CouponValidateResponse`	Validate coupon eligibility & calculate discount	Authenticated

5. Coupon Validation Logic

When a passenger enters a coupon code during checkout, `POST /api/fares/coupons/validate` executes the following validation pipeline:

1. Lookup coupon by uppercase code.
2. Verify status is `ACTIVE`.
3. Verify current date is within `[validFrom, validUntil]`.
4. Verify `usageCount < usageLimit`.

5. Verify order subtotal >= `minFare`.
6. Compute discount: `min(subtotal * (discountPercentage / 100), maxDiscount)`.
7. Return calculated discount and new payable total.

CHAPTER 5.8

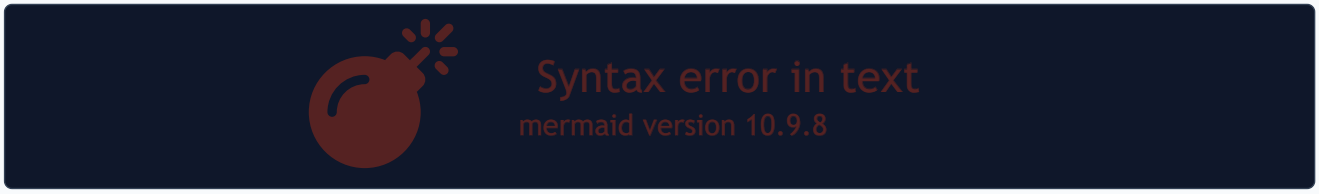
Booking Service Documentation

1. Executive Summary & Responsibility

The **Booking Service** (`booking-service`) orchestrates the flight ticket reservation lifecycle. It acts as the operational core of customer transactions, generating unique 6-character aviation Passenger Name Records (PNRs), validating flight and seat availability via OpenFeign, computing ticket prices with Fare Service, persisting passenger manifest rosters, publishing seat deduction events to RabbitMQ, and managing booking cancellations.

- **Service Name**: `booking-service`
- **Default Port**: `8088`
- **Application Context**: `/`
- **Base URI Path**: `/api/bookings`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_booking_db` (MySQL)
- **Inter-Service Communication**:
  - OpenFeign: `FlightServiceClient` (`http://FLIGHT-SERVICE`)
  - OpenFeign: `FareServiceClient` (`http://FARE-SERVICE`)
- **Messaging**: RabbitMQ producer publishing `BookingCreatedEvent` to `booking-created` queue
- **Service Discovery**: Registered in Eureka as `BOOKING-SERVICE`

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_booking_db.bookings`

Stores confirmed and cancelled airline reservations with unique PNR keys.

Column	Type	Constraints	Description
`booking_id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Reservation entity ID
`pnr`	`VARCHAR(20)`	NOT NULL, UNIQUE	6-character aviation Passenger Name Record
`user_id`	`BIGINT`	NOT NULL	ID of passenger user who placed booking
`flight_id`	`BIGINT`	NOT NULL	Commercial flight ID
`total_fare`	`DECIMAL(10,2)`	NOT NULL	Final charged ticket total
`status`	`VARCHAR(20)`	NOT NULL, DEFAULT `'CONFIRMED'`	`CONFIRMED` , `CANCELLED` , `PENDING`
`booking_date`	`DATETIME`	NOT NULL	Reservation creation timestamp

3.2 `aerobook_booking_db.passengers`

Stores individual passenger demographics and seat assignments linked to the reservation.

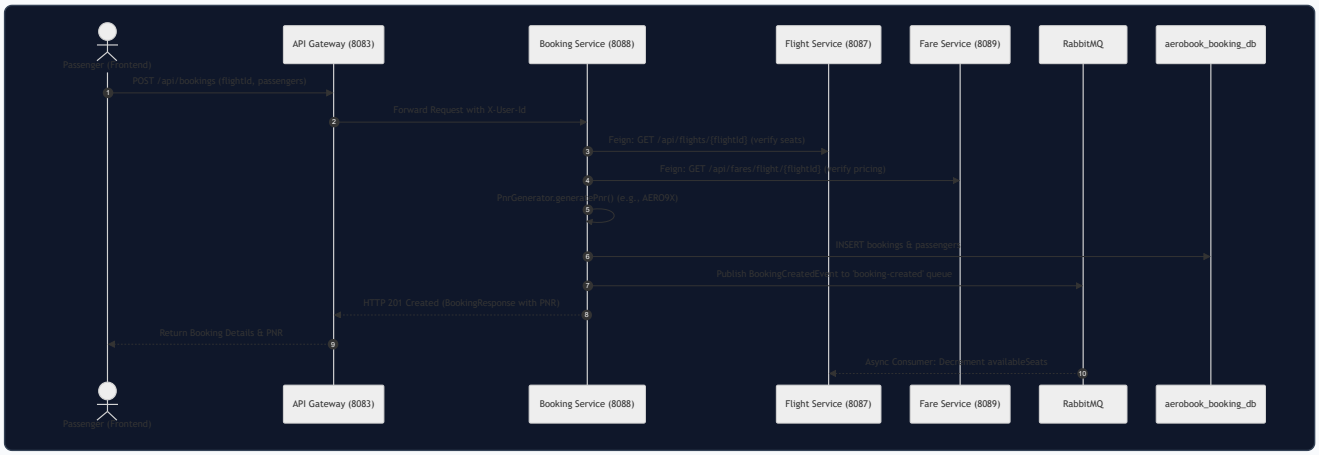
Column	Type	Constraints	Description
`passenger_id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Passenger roster ID
`booking_id`	`BIGINT`	NOT NULL, FOREIGN KEY (`bookings.booking_id`)	Parent reservation ID
`first_name`	`VARCHAR(50)`	NOT NULL	Passenger legal first name
`last_name`	`VARCHAR(50)`	NOT NULL	Passenger legal surname
`gender`	`VARCHAR(10)`	NOT NULL	Passenger gender

Column	Type	Constraints	Description
`age`	`INT`	NOT NULL	Age at travel time
`seat_number`	`VARCHAR(10)`	NULL	Assigned cabin seat (e.g. `12A`, `14B`)

4. REST API Endpoint Catalog

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/bookings/health`	*None*	`String`	Health check probe	Public
`POST`	`/api/bookings`	`BookingRequest`	`BookingResponse` (HTTP 201)	Create new flight reservation and passengers	Authenticated
`GET`	`/api/bookings`	*None*	`List<BookingResponse>`	Retrieve all bookings across platform	Admin / Staff
`GET`	`/api/bookings/{bookingId}`	Path: `bookingId`	`BookingResponse`	Get booking details by ID	Owner / Admin / Staff
`GET`	`/api/bookings/pnr/{pnr}`	Path: `pnr`	`BookingResponse`	Retrieve reservation details by 6-char PNR	Authenticated / Staff
`GET`	`/api/bookings/user/{userId}`	Path: `userId`	`List<BookingResponse>`	Get all reservations placed by a user	Owner / Admin
`GET`	`/api/bookings/flight/{flightId}`	Path: `flightId`	`List<BookingResponse>`	Get all bookings on a specific flight	Admin / Staff
`DELETE`	`/api/bookings/{bookingId}`	Path: `bookingId`	`Void` (HTTP 204)	Cancel reservation	Owner / Admin

5. Booking Creation & PNR Generation Workflow



CHAPTER 5.9

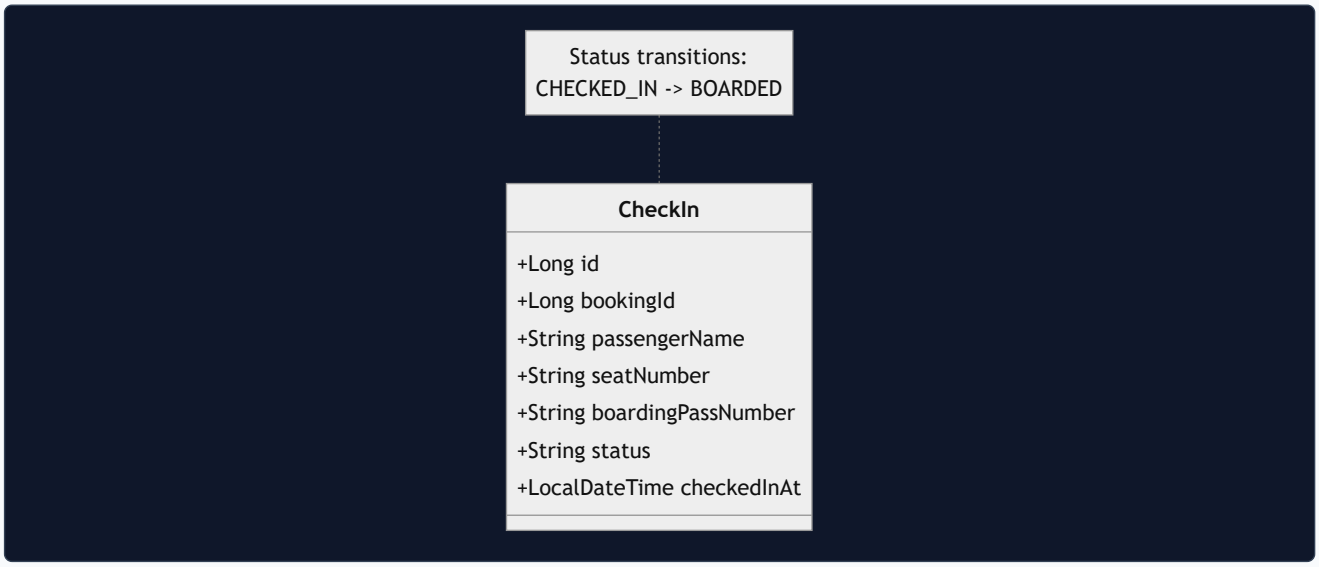
Check-In Service Documentation

1. Executive Summary & Responsibility

The **Check-In Service** (`check-in-service`) manages passenger flight check-in, seat confirmation, boarding pass generation, departure gate boarding verification, and airport operations auditing. It bridges the transition from a confirmed reservation (`booking-service`) to physical aircraft boarding, generating verifiable boarding pass identifiers (format `BP-<bookingId>-<seatNumber>`) and updating passenger statuses during departure gate boarding scans.

- **Service Name**: `check-in-service`
- **Default Port**: `8090`
- **Application Context**: `/`
- **Base URI Path**: `/api/check-ins`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_checkin_db` (MySQL)
- **Role Enforcement**: Consumes `X-User-Role` forwarded by `api-gateway`
- **Service Discovery**: Registered in Eureka as `CHECK-IN-SERVICE`

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_checkin_db.check_in`

Stores passenger check-in verification records and boarding credentials.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Check-in verification record ID
`booking_id`	`BIGINT`	NOT NULL	Associated reservation ID in `booking-service`
`passenger_name`	`VARCHAR(100)`	NOT NULL	Passenger full legal name
`seat_number`	`VARCHAR(10)`	NOT NULL	Confirmed cabin seat (e.g. `12A` , `15C`)
`boarding_pass_number`	`VARCHAR(50)`	NOT NULL, UNIQUE	Aviation barcode pass (e.g. `BP-1-12A`)
`status`	`VARCHAR(20)`	NOT NULL, DEFAULT `'CHECKED_IN'`	Lifecycle state: `CHECKED_IN` , `BOARDED`
`checked_in_at`	`DATETIME`	NOT NULL	Timestamp of digital check-in

4. REST API Endpoint Catalog

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/check-ins/health`	*None*	`Map<String, Object>`	Health check & operational metrics probe	Public
`POST`	`/api/check-ins`	`CheckInRequest` (`bookingId` , `passengerName` , `seatNumber`)	`CheckIn` entity (HTTP 200)	Perform passenger check-in & issue boarding pass	Authenticated
`GET`	`/api/check-ins/booking/{bookingId}`	Path: `bookingId`	`CheckIn` entity	Retrieve check-in record for booking ID	Authenticated
`GET`	`/api/check-ins/{id}`	Path: `id`	`CheckIn` entity	Retrieve check-in record by primary ID	Authenticated
`GET`	`/api/check-ins`	Header: `X-User-Role`	`List<CheckIn>`	List all check-ins (manifest directory)	Admin / Staff
`POST`	`/api/check-ins/verify`	Param: `boardingPassNumber` Header: `X-User-Role`	`CheckIn` entity	Airport gate boarding verification (`BOARDED`)	Staff / Admin

5. Boarding Pass Lifecycle & Airport Gate Verification

1. **Self-Service Check-In**:
   - Passenger submits `POST /api/check-ins` with `bookingId` , `passengerName` , and chosen `seatNumber` .
   - The service verifies booking eligibility, saves the record with status `CHECKED_IN` , and generates `boardingPassNumber` = `BP-{bookingId}-{seatNumber}` .
   - Triggers downstream notification with generated PDF boarding pass via `notification-service` .
2. **Airport Departure Gate Scan**:
   - At boarding, airline staff scans the physical or mobile boarding pass barcode.
   - Staff frontend calls `POST /api/check-ins/verify?boardingPassNumber=BP-1-12A` .
   - Service verifies existence, transitions status to `BOARDED` , and logs gate clearance.

CHAPTER 5.10

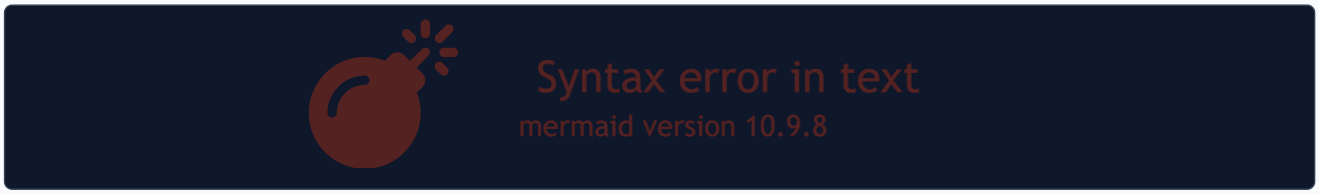
Notification Service Documentation

1. Executive Summary & Responsibility

The **Notification Service** (`notification-service`) manages outbound email communications, on-the-fly PDF document generation (e-tickets and boarding passes), and persistent in-app notifications. It provides transactional notifications for user onboarding, booking confirmations, airport check-in receipts, flight schedule alterations, payment refunds, and administrative platform broadcasts.

- **Service Name**: `notification-service`
- **Default Port**: `8091`
- **Application Context**: `/`
- **Base URI Path**: `/api/notifications`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_notification_db` (MySQL)
- **Document Engine**: OpenPDF (fork of iText) for vector PDF e-tickets & boarding passes
- **Email Delivery Engine**: Spring `JavaMailSender` (SMTP) with resilient HTML templates and development logging fallback
- **Service Discovery**: Registered in Eureka as `NOTIFICATION-SERVICE`

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_notification_db.notifications`

Stores persistent in-app notification records and delivery receipts.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Notification entity ID
`user_id`	`BIGINT`	NULL	Recipient user ID (NULL for global broadcasts)
`recipient_email`	`VARCHAR(150)`	NULL	Recipient email address
`title`	`VARCHAR(255)`	NOT NULL	Brief headline / subject
`message`	`TEXT`	NOT NULL	Notification body content
`type`	`VARCHAR(50)`	NOT NULL	Category enum (`NotificationType`)
`status`	`VARCHAR(30)`	NOT NULL	Delivery state (`NotificationStatus`)
`flight_number`	`VARCHAR(50)`	NULL	Associated flight number (if flight-specific)
`created_at`	`DATETIME`	NOT NULL	Notification generation timestamp
`read_at`	`DATETIME`	NULL	Timestamp when user opened notification

4. REST API Endpoint Catalog

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/notifications/health`	*None*	`String`	Health check probe	Public
`POST`	`/api/notifications/send`	`NotificationRequest`	`NotificationResponse` (HTTP 201)	Save and send an in-app notification	Internal / Services

Method	Path	Request Body / Params	Response Body	Description	Access
`POST`	`/api/notifications/email/booking`	`BookingNotificationRequest`	`NotificationResponse`	Send booking confirmation email with attached ticket PDF	Internal / Booking
`POST`	`/api/notifications/email/checkin`	`CheckInNotificationRequest`	`NotificationResponse`	Send check-in email with boarding pass PDF	Internal / Check-in
`POST`	`/api/notifications/email/flight-alert`	`FlightAlertNotificationRequest`	`NotificationResponse`	Alert passengers of flight delays or gate changes	Internal / Staff
`POST`	`/api/notifications/email/refund`	`RefundNotificationRequest`	`NotificationResponse`	Send payment refund notification email	Internal / Payment
`POST`	`/api/notifications/email/verify`	`EmailVerificationRequest`	`NotificationResponse`	Send registration 6-digit OTP code	Internal / Auth
`POST`	`/api/notifications/email/reset-password`	`EmailVerificationRequest`	`NotificationResponse`	Send password reset verification code	Internal / Auth
`POST`	`/api/notifications/broadcast`	`BroadcastRequest`	`NotificationResponse`	Broadcast alert to all active users	Admin
`GET`	`/api/notifications/user/{userId}`	Path: `userId`	`List<NotificationResponse>`	Get user's in-app notification inbox	Authenticated
`PUT`	`/api/notifications/{id}/read`	Path: `id`	`NotificationResponse`	Mark single notification as read	Authenticated
`PUT`	`/api/notifications/user/{userId}/read-all`	Path: `userId`	`Void` (HTTP 200)	Mark all user notifications as read	Authenticated
`GET`	`/api/notifications/unread-count/{userId}`	Path: `userId`	`Long`	Count unread notifications for badge counter	Authenticated
`GET`	`/api/notifications`	*None*	`List<NotificationResponse>`	List all platform notifications	Admin
`GET`	`/api/notifications/pdf/ticket`	Query: `pnr`, `flightNumber`, etc.	`byte[]` (`application/pdf`)	Generate on-the-fly e-ticket PDF download	Authenticated
`GET`	`/api/notifications/pdf/boarding-pass`	Query: `boardingPassNumber`, etc.	`byte[]` (`application/pdf`)	Generate on-the-fly boarding pass PDF download	Authenticated

5. PDF Generation & Email Dispatch Engine

1. **Dynamic PDF Generation**:
   - Built on `OpenPDF` (com.github.librepdf:openpdf:2.0.3).

- Generates vector-quality airline e-tickets and boarding passes with barcode placeholders, seat allocations, flight numbers, and passenger details.
- Streams directly to HTTP client response (`application/pdf`) or attaches to outgoing MIME multipart email.

2. **Resilient Email Dispatching**:
   - Configured via `spring.mail.host` , `spring.mail.port` , `spring.mail.username` , and `spring.mail.password` .
   - When SMTP credentials are unconfigured or in offline development, the service falls back to structured logging without failing the upstream business transaction.

CHAPTER 5.11

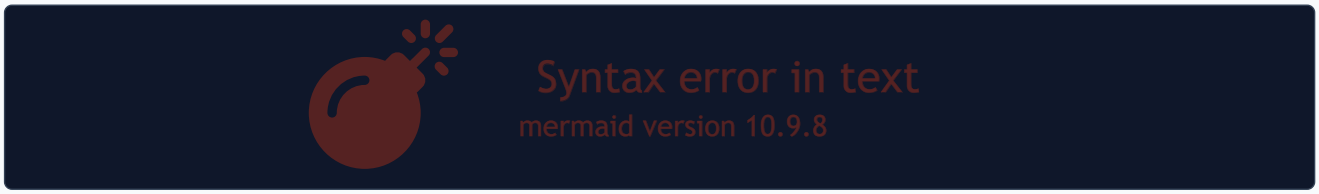
Payment Service Documentation

1. Executive Summary & Responsibility

The **Payment Service** (`payment-service`) integrates AeroBook with Razorpay payment gateways to process secure online ticket purchases, cryptographic signature verifications, and automated refunds. It manages the complete financial transaction lifecycle, translating booking amounts into Indian Paise (`amount * 100`), verifying HMAC-SHA256 signatures, handling asynchronous webhook notifications, and executing partial or full refunds upon booking cancellation.

- **Service Name**: `payment-service`
- **Default Port**: `8092`
- **Application Context**: `/`
- **Base URI Path**: `/api/payments`
- **Spring Boot Version**: `3.3.4` (Java 21)
- **Database Schema**: `aerobook_payment_db` (MySQL)
- **Payment Gateway**: Razorpay API (`razorpay-java:1.4.3`) with HMAC-SHA256 verification
- **Service Discovery**: Registered in Eureka as `PAYMENT-SERVICE`

2. Architecture & Domain Model



3. Database Schema & Tables

3.1 `aerobook_payment_db.payments`

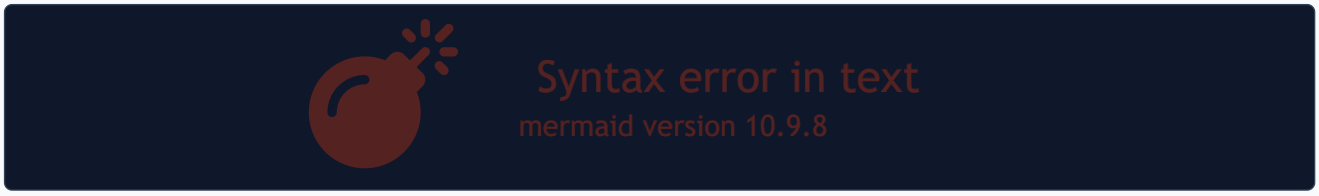
Stores transaction records, gateway order references, and cryptographic proofs.

Column	Type	Constraints	Description
`id`	`BIGINT`	PRIMARY KEY, AUTO_INCREMENT	Payment entity ID
`booking_id`	`BIGINT`	NOT NULL	Associated reservation ID in `booking-service`
`pnr`	`VARCHAR(20)`	NULL	Associated booking PNR
`order_id`	`VARCHAR(100)`	NOT NULL, UNIQUE	Razorpay Order ID (e.g. `order_M1bC7x...`)
`payment_id`	`VARCHAR(100)`	NULL	Razorpay Payment ID (e.g. `pay_N8eR4z...`)
`amount`	`DOUBLE`	NOT NULL	Transaction amount in standard currency (INR)
`amount_in_paise`	`BIGINT`	NOT NULL	Amount in subunit Paise (`amount * 100`)
`currency`	`VARCHAR(10)`	NOT NULL, DEFAULT `'INR'`	Currency code
`receipt`	`VARCHAR(100)`	NULL	Internal receipt / invoice reference
`status`	`VARCHAR(20)`	NOT NULL, DEFAULT `'CREATED'`	`CREATED` , `AUTHORIZED` , `CAPTURED` , `REFUNDED` , `FAILED`
`signature`	`VARCHAR(255)`	NULL	Cryptographic HMAC-SHA256 payment signature
`refund_id`	`VARCHAR(100)`	NULL	Razorpay Refund ID upon cancellation
`refund_amount`	`DOUBLE`	NULL	Refunded sum in INR
`created_at`	`DATETIME`	NOT NULL	Payment creation timestamp
`updated_at`	`DATETIME`	NULL	Last status update timestamp

4. REST API Endpoint Catalog

Method	Path	Request Body / Params	Response Body	Description	Access
`GET`	`/api/payments/health`	*None*	`String`	Health check probe	Public
`POST`	`/api/payments/orders`	`PaymentOrderRequest`	`PaymentOrderResponse` (HTTP 201)	Create Razorpay order ID for booking	Authenticated
`POST`	`/api/payments/verify`	`PaymentVerifyRequest`	`PaymentVerifyResponse`	Verify HMAC signature & capture payment	Authenticated
`POST`	`/api/payments/webhook`	Webhook Payload & Header: `X-Razorpay-Signature`	`ResponseEntity`	Asynchronous Razorpay webhook receiver	Gateway / Webhook
`POST`	`/api/payments/{id}/refund`	`PaymentRefundRequest`	`PaymentRefundResponse`	Initiate partial or full ticket refund	Authenticated / Staff
`GET`	`/api/payments`	*None*	`List<PaymentRecord>`	List all payment transactions across platform	Admin / Staff
`GET`	`/api/payments/{id}`	Path: `id`	`PaymentRecord`	Retrieve payment record by ID	Authenticated / Admin
`GET`	`/api/payments/booking/{bookingId}`	Path: `bookingId`	`PaymentRecord`	Retrieve payment record by booking ID	Authenticated / Admin

5. Razorpay Checkout & Cryptographic Verification Workflow



CHAPTER 6

Frontend Application Documentation

1. Executive Summary & Architecture Overview

The **AeroBook Frontend** (`frontend`) is a modern, responsive Single Page Application (SPA) built with **Angular 18**. It utilizes Angular's latest architectural patterns: **Standalone Components**, **Angular Signals** for reactive fine-grained state management, and a custom atomic design component system (`ab-*`). The frontend communicates exclusively with the backend through the Spring Cloud API Gateway (`http://localhost:8083`), enforcing a strict Single Entry Point architecture.

- **Framework**: Angular `18.2.0`
- **Language**: TypeScript `5.5.4`
- **State Management**: Angular Signals (`signal()` , `computed()`)
- **Component Architecture**: 100% Standalone Components (zero NgModules)
- **Styling Architecture**: Custom CSS Design Tokens with dual-theme engine (`light` / `dark`)
- **API Target**: `http://localhost:8083` (API Gateway)
- **Default Port**: `4200` (`ng serve`)

2. Directory Layout & Module Structure

```
frontend/src/app/
├── app.component.ts              # Root component container
├── app.config.ts                 # Application providers (HttpClient, Router, Interceptors)
├── app.routes.ts                 # Top-level client route definitions & guards
├── core/
│   ├── api/                      # Typed REST API service clients
│   ├── guards/                   # Functional route guards (customerGuard, staffGuard, adminGuard)
│   ├── interceptors/             # HTTP Auth token injection & error handlers
│   ├── models/                   # TypeScript interfaces & domain models
│   ├── services/                 # Core singleton services (AuthService, BookingService, etc.)
│   └── theme/                    # ThemeService & design token managers
├── features/
│   ├── home/                     # Public landing page, flight search, booking teaser
│   ├── customer/                 # Passenger dashboard (my bookings, check-in, profile)
│   ├── staff/                    # Airport staff dashboard (gate verify, passenger lookup, incidents)
│   └── admin/                    # Admin operations (fleet, flights, fares, coupons, analytics)
└── shared/
    ├── components/               # Composite UI modules (navbar, login-modal, register-modal)
    └── ui/                       # Atomic "ab-*" UI Component Library (20+ components)
```

3. UI Component Library (`ab-*`)

The application features a built-in atomic component library prefixed with `ab-` , designed for aviation UI workflows and consistent accessibility:

Component	Selector	Purpose
`ab-button`	`<ab-button>`	Primary, secondary, outline, ghost, and danger button variants
`ab-card`	`<ab-card>`	Structured content container with header, body, footer slots
`ab-hero-card`	`<ab-hero-card>`	Hero highlight section with flight imagery and CTA
`ab-stat-card`	`<ab-stat-card>`	Metric card for dashboard stats (revenue, active flights)
`ab-ticket`	`<ab-ticket>`	Realistic perforated boarding pass / e-ticket card display
`ab-table`	`<ab-table>`	Data table with sorting, pagination, and action slots
`ab-modal`	`<ab-modal>`	Accessible dialog modal backdrop with keyboard traps
`ab-input`	`<ab-input>`	Validated input field with floating labels and icons
`ab-select`	`<ab-select>`	Custom styled dropdown select
`ab-checkbox`	`<ab-checkbox>`	Accessible checkbox input
`ab-toggle`	`<ab-toggle>`	Switch toggle (e.g., active coupon status)
`ab-segmented`	`<ab-segmented>`	Segmented button group (e.g. cabin class tabs)
`ab-status-pill`	`<ab-status-pill>`	Colored badge (`CONFIRMED` , `ON_TIME` , `DELAYED` , `OPEN`)
`ab-meter`	`<ab-meter>`	Capacity progress bar for aircraft seats
`ab-spark`	`<ab-spark>`	Micro sparkline for trend visualization
`ab-skeleton`	`<ab-skeleton>`	Shimmer placeholder during asynchronous data loading

Component	Selector	Purpose
`ab-empty-state`	`<ab-empty-state>`	Placeholder when query returns zero records
`ab-error-state`	`<ab-error-state>`	User-friendly error message container with retry action

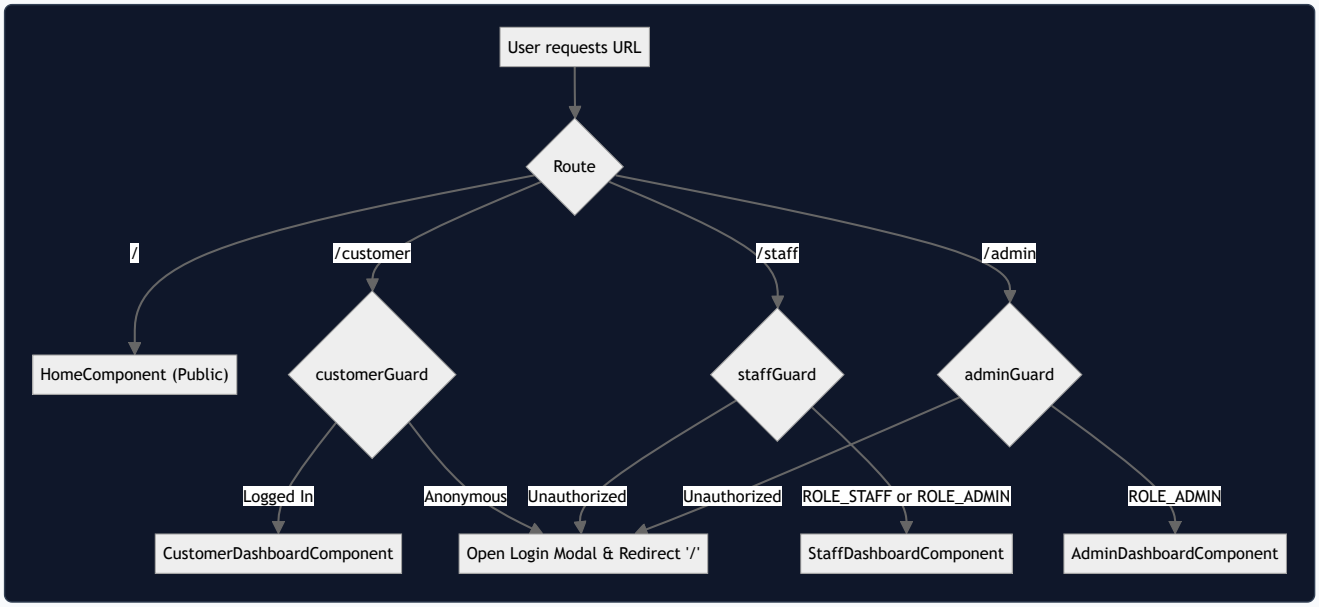
4. State Management with Angular Signals

The frontend replaces RxJS Subject-heavy stores with lightweight, reactive Angular Signals:

- **`AuthService` Signals**:
  - `tokenSignal = signal<string | null>(...)`
  - `roleSignal = signal<RoleType | null>(...)`
  - `isAuthenticated = computed(() => !!this.tokenSignal())`
  - `isAdmin = computed(() => ...)`
  - `isStaff = computed(() => ...)`
  - `isCustomer = computed(() => !this.isAdmin() && !this.isStaff())`
  - `loginModalOpen = signal<boolean>(false)`
  - `registerModalOpen = signal<boolean>(false)`
- **`ThemeService` Signals**:
  - `theme = signal<'light' | 'dark'>(...)`
  - `isDark = computed(() => this.theme() === 'dark')`
  - Automatically updates root `<html>` attribute `data-theme` and synchronizes with `localStorage`.

5. Client-Side Routing & Route Protection

Protected routes are secured using functional Angular CanActivateFn guards:



6. HTTP Interceptors & Token Propagation

1. **JWT Auth Interceptor**:
   - Intercepts all outgoing requests directed at `http://localhost:8083`.
   - Inspects `AuthService.currentToken()`.
   - Appends `Authorization: Bearer <jwt-token>` header automatically.
2. **Error Interceptor**:
   - Catches HTTP `401 Unauthorized` responses: Clears authentication state, prompts the login modal, and redirects to home.
   - Catches HTTP `403 Forbidden` responses: Displays unauthorized warning notifications.
   - Catches HTTP `500` / connection failures: Renders `ab-error-state` with retry logic.

CHAPTER 7

Master API Reference Catalog

This document provides the definitive, unified REST API reference for the entire AeroBook microservices ecosystem. All requests originate through the **Spring Cloud API Gateway** (`http://localhost:8083`), which routes them dynamically to downstream services.

1. Global Gateway Overview Endpoints (`api-gateway`)

Method	Public Gateway Path	Downstream Route	Access	Description
`GET`	`/api/gateway/health`	Local Gateway	Public	Gateway operational status & uptime
`GET`	`/api/gateway/services`	Local Gateway	Public	Discovered microservices registry map
`GET`	`/api/gateway/admin/summary`	Local Gateway	Public	Platform-wide operational stats
`GET`	`/api/gateway/endpoints`	Local Gateway	Public	Catalog of all route endpoints
`GET`	`/api/gateway/rbac-matrix`	Local Gateway	Public	Platform RBAC permissions matrix
`GET`	`/api/gateway/presentation-demo`	Local Gateway	Public	Demo telemetry for investor / tech walkthroughs

2. Authentication & Credential Endpoints (`auth-service`)

Downstream Service Port: `8082` | Target Route: `lb://AUTH-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/auth/health`	*None*	200 OK	Auth service health status	Public
`POST`	`/api/auth/send-verification-code`	`{ email }`	200 OK	Dispatch 6-digit OTP code to email	Public
`POST`	`/api/auth/verify-code`	`{ email, code }`	200 OK	Validate registration OTP code	Public
`POST`	`/api/auth/forgot-password`	`{ email }`	200 OK	Dispatch password recovery code	Public
`POST`	`/api/auth/reset-password`	`{ email, code, newPassword }`	200 OK	Reset account password	Public
`POST`	`/api/auth/register`	`{ email, password, firstName, lastName, ... }`	201 Created	Passenger registration	Public
`POST`	`/api/auth/register-admin`	`{ email, password, firstName, lastName, setupKey }`	201 Created	Admin registration (Setup key required)	Restricted
`POST`	`/api/auth/register-staff`	`{ email, password, firstName, lastName, setupKey }`	201 Created	Staff registration (Setup key required)	Restricted
`POST`	`/api/auth/login`	`{ email, password }`	200 OK	Authenticate and obtain JWT token	Public
`POST`	`/api/auth/refresh-token`	`{ refreshToken }`	200 OK	Exchange refresh token for fresh JWT	Authenticated
`GET`	`/api/auth/credentials`	*None*	200 OK	List all credentials (passwords excluded)	Admin
`PUT`	`/api/auth/credentials/{userId}/role`	`{ role }`	200 OK	Update user role assignment	Admin

3. User & Staff Incident Endpoints (`user-service`)

Downstream Service Port: `8084` | Target Route: `lb://USER-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/v1/users/health`	*None*	200 OK	User service health status	Public
`POST`	`/api/v1/users`	`UserRequest`	201 Created	Create user profile record	Internal / Auth
`GET`	`/api/v1/users`	*None*	200 OK	Retrieve all user profiles	Admin / Staff
`GET`	`/api/v1/users/{id}`	Path: `id`	200 OK	Get user profile by ID	Authenticated

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/v1/users/email/{email}`	Path: `email`	200 OK	Get user profile by email	Authenticated
`PUT`	`/api/v1/users/{id}`	`UserRequest`	200 OK	Update user profile	Owner / Admin
`DELETE`	`/api/v1/users/{id}`	Path: `id`	204 No Content	Delete user account	Admin
`GET`	`/api/v1/users/incidents`	*None*	200 OK	List all passenger incident reports	Admin / Staff
`GET`	`/api/v1/users/incidents/{id}`	Path: `id`	200 OK	Get single incident ticket	Admin / Staff
`POST`	`/api/v1/users/incidents`	`IncidentReportRequest`	201 Created	File operational passenger incident	Staff / Admin
`PUT`	`/api/v1/users/incidents/{id}/status`	Query: `status`	200 OK	Update incident workflow state	Staff / Admin
`DELETE`	`/api/v1/users/incidents/{id}`	Path: `id`	204 No Content	Delete incident report	Admin
`GET`	`/api/v1/users/incidents/count/open`	*None*	200 OK	Count unresolved incidents	Staff / Admin

4. Flights & Fleet Endpoints (`flight-service`)

Downstream Service Port: `8087` | Target Route: `lb://FLIGHT-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/flights/health`	*None*	200 OK	Flight service health status	Public
`GET`	`/api/flights/search`	`source` , `destination` , `date`	200 OK	Search flights by origin/dest/date	Public
`GET`	`/api/flights`	*None*	200 OK	List all scheduled flights	Public
`GET`	`/api/flights/{id}`	Path: `id`	200 OK	Get flight details & seat availability	Public
`POST`	`/api/admin/flights`	`CreateFlightRequest`	201 Created	Schedule a new flight	Admin
`PUT`	`/api/admin/flights/{id}`	`UpdateFlightRequest`	200 OK	Modify flight schedule/status	Admin
`DELETE`	`/api/admin/flights/{id}`	Path: `id`	204 No Content	Cancel / delete scheduled flight	Admin
`GET`	`/api/admin/flights`	*None*	200 OK	Administrative flight roster	Admin / Staff
`POST`	`/api/admin/aircrafts`	`CreateAircraftRequest`	201 Created	Add new aircraft to fleet	Admin
`GET`	`/api/admin/aircrafts`	*None*	200 OK	List all fleet aircraft	Admin / Staff
`GET`	`/api/admin/aircrafts/{id}`	Path: `id`	200 OK	Get single aircraft specs	Admin / Staff
`DELETE`	`/api/admin/aircrafts/{id}`	Path: `id`	204 No Content	Decommission aircraft	Admin

5. Pricing & Coupon Endpoints (`fare-service`)

Downstream Service Port: `8089` | Target Route: `lb://FARE-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/fares/health`	*None*	200 OK	Fare service health status	Public
`POST`	`/api/fares`	`FareRequest`	201 Created	Define cabin fares for a flight	Admin
`GET`	`/api/fares/{id}`	Path: `id`	200 OK	Get fare record by ID	Public
`GET`	`/api/fares/flight/{flightId}`	Path: `flightId`	200 OK	Get cabin pricing for flight	Public
`GET`	`/api/fares`	*None*	200 OK	List all flight fare rules	Public / Admin
`PUT`	`/api/fares/{id}`	`FareRequest`	200 OK	Update pricing tiers & tax	Admin
`DELETE`	`/api/fares/{id}`	Path: `id`	204 No Content	Delete fare record	Admin
`GET`	`/api/fares/coupons`	*None*	200 OK	List all promotional coupons	Public / Admin
`GET`	`/api/fares/coupons/{id}`	Path: `id`	200 OK	Get coupon terms by ID	Public / Admin
`POST`	`/api/fares/coupons`	`CouponRequest`	201 Created	Create new promo coupon	Admin
`PUT`	`/api/fares/coupons/{id}`	`CouponRequest`	200 OK	Update coupon parameters	Admin
`PUT`	`/api/fares/coupons/{id}/toggle`	Path: `id`	200 OK	Toggle coupon ACTIVE/INACTIVE	Admin
`DELETE`	`/api/fares/coupons/{id}`	Path: `id`	204 No Content	Delete coupon code	Admin

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`POST`	`/api/fares/coupons/validate`	`CouponValidateRequest`	200 OK	Validate coupon eligibility	Authenticated

6. Bookings & Passenger Endpoints (`booking-service`)

Downstream Service Port: `8088` | Target Route: `lb://BOOKING-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/bookings/health`	*None*	200 OK	Booking service health status	Public
`POST`	`/api/bookings`	`BookingRequest`	201 Created	Create booking & assign PNR	Authenticated
`GET`	`/api/bookings`	*None*	200 OK	List all platform bookings	Admin / Staff
`GET`	`/api/bookings/{bookingId}`	Path: `bookingId`	200 OK	Get booking details by ID	Owner / Admin / Staff
`GET`	`/api/bookings/pnr/{pnr}`	Path: `pnr`	200 OK	Lookup booking by 6-char PNR	Authenticated / Staff
`GET`	`/api/bookings/user/{userId}`	Path: `userId`	200 OK	List bookings placed by user	Owner / Admin
`GET`	`/api/bookings/flight/{flightId}`	Path: `flightId`	200 OK	List bookings on a flight	Admin / Staff
`DELETE`	`/api/bookings/{bookingId}`	Path: `bookingId`	204 No Content	Cancel flight reservation	Owner / Admin

7. Check-In & Boarding Gate Endpoints (`check-in-service`)

Downstream Service Port: `8090` | Target Route: `lb://CHECK-IN-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/check-ins/health`	*None*	200 OK	Check-in service health probe	Public
`POST`	`/api/check-ins`	`CheckInRequest`	200 OK	Check in & issue boarding pass	Authenticated
`GET`	`/api/check-ins/booking/{bookingId}`	Path: `bookingId`	200 OK	Get check-in by booking ID	Authenticated
`GET`	`/api/check-ins/{id}`	Path: `id`	200 OK	Get check-in by record ID	Authenticated
`GET`	`/api/check-ins`	Header: `X-User-Role`	200 OK	Manifest audit list	Admin / Staff
`POST`	`/api/check-ins/verify`	Param: `boardingPassNumber`	200 OK	Airport gate boarding verification	Staff / Admin

8. Notifications & PDF Endpoints (`notification-service`)

Downstream Service Port: `8091` | Target Route: `lb://NOTIFICATION-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/notifications/health`	*None*	200 OK	Notification service health probe	Public
`POST`	`/api/notifications/send`	`NotificationRequest`	201 Created	Dispatch in-app alert	Internal
`POST`	`/api/notifications/email/booking`	`BookingNotificationRequest`	200 OK	Send booking confirmation email + PDF	Internal
`POST`	`/api/notifications/email/checkin`	`CheckInNotificationRequest`	200 OK	Send check-in email + boarding pass	Internal
`POST`	`/api/notifications/email/flight-alert`	`FlightAlertNotificationRequest`	200 OK	Dispatch flight delay/gate alert	Internal
`POST`	`/api/notifications/email/refund`	`RefundNotificationRequest`	200 OK	Send refund confirmation email	Internal
`POST`	`/api/notifications/email/verify`	`EmailVerificationRequest`	200 OK	Send registration OTP code	Internal
`POST`	`/api/notifications/email/reset-password`	`EmailVerificationRequest`	200 OK	Send password reset code	Internal
`POST`	`/api/notifications/broadcast`	`BroadcastRequest`	200 OK	Global notification broadcast	Admin
`GET`	`/api/notifications/user/{userId}`	Path: `userId`	200 OK	In-app notification inbox	Authenticated
`PUT`	`/api/notifications/{id}/read`	Path: `id`	200 OK	Mark notification as read	Authenticated
`PUT`	`/api/notifications/user/{userId}/read-all`	Path: `userId`	200 OK	Mark all notifications read	Authenticated

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/notifications/unread-count/{userId}`	Path: `userId`	200 OK	Unread notification count badge	Authenticated
`GET`	`/api/notifications`	*None*	200 OK	List all notifications	Admin
`GET`	`/api/notifications/pdf/ticket`	Query parameters	200 OK	Download e-ticket PDF	Authenticated
`GET`	`/api/notifications/pdf/boarding-pass`	Query parameters	200 OK	Download boarding pass PDF	Authenticated

9. Payments & Refunds Endpoints (`payment-service`)

Downstream Service Port: `8092` | Target Route: `lb://PAYMENT-SERVICE`

Method	Endpoint Path	Payload / Query	Response Code	Description	Access
`GET`	`/api/payments/health`	*None*	200 OK	Payment service health probe	Public
`POST`	`/api/payments/orders`	`PaymentOrderRequest`	201 Created	Create Razorpay order	Authenticated
`POST`	`/api/payments/verify`	`PaymentVerifyRequest`	200 OK	Verify signature & capture payment	Authenticated
`POST`	`/api/payments/webhook`	Webhook body	200 OK	Asynchronous Razorpay webhook	Gateway
`POST`	`/api/payments/{id}/refund`	`PaymentRefundRequest`	200 OK	Initiate Razorpay refund	Staff / Admin
`GET`	`/api/payments`	*None*	200 OK	List all transaction records	Admin / Staff
`GET`	`/api/payments/{id}`	Path: `id`	200 OK	Get payment record by ID	Authenticated
`GET`	`/api/payments/booking/{bookingId}`	Path: `bookingId`	200 OK	Get payment for booking	Authenticated

CHAPTER 8

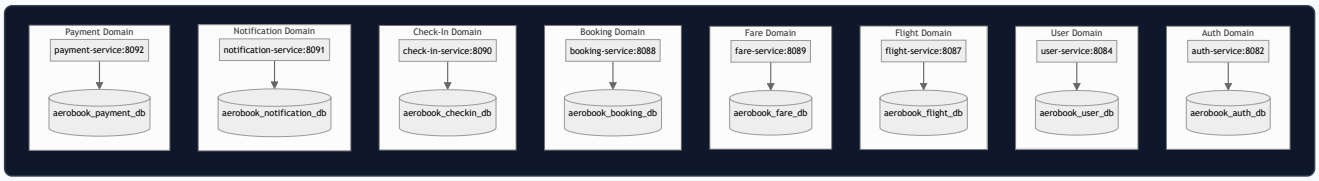
Database Design & Persistence Architecture

1. Architectural Philosophy: Database-Per-Service

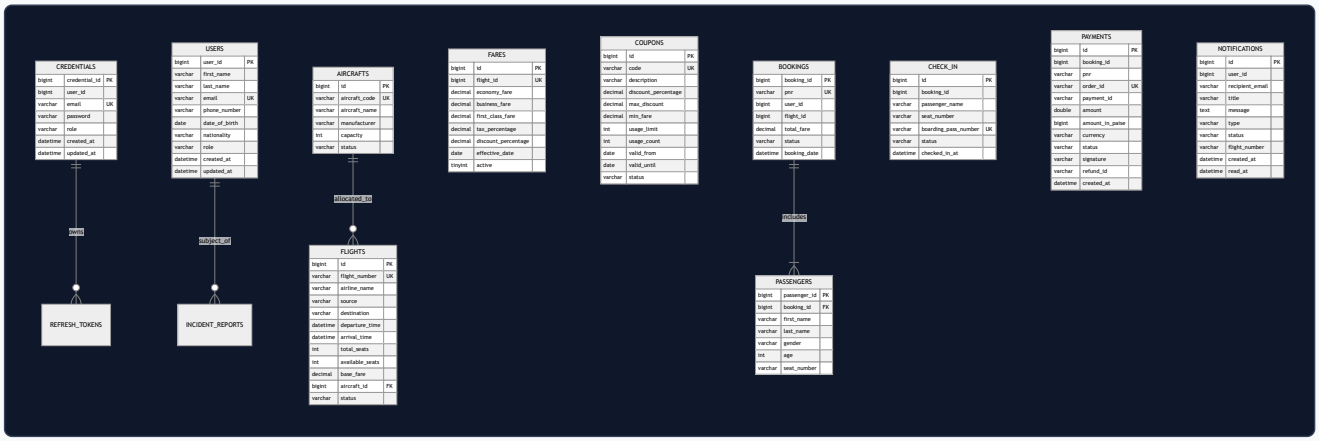
AeroBook strictly adheres to the **Database-per-Service** architectural pattern. Each business domain microservice owns its private, isolated MySQL database schema. No microservice is permitted to execute direct SQL queries, joins, or mutations against a database owned by another microservice.

All cross-domain queries and data synchronizations are orchestrated via:

1. **Synchronous OpenFeign REST calls** for operational dependencies (e.g., Booking Service querying Flight Service for seat availability).
2. **Asynchronous RabbitMQ AMQP messaging** for state propagation (e.g., Booking Created events reducing flight seat inventory).
3. **Identity Token Claims** for user context propagation (`X-User-Id` , `X-User-Role`).



2. Complete Entity-Relationship Diagrams (ERD)



3. Database Schemas & Table DDL Reference

3.1 `aerobook_auth_db`

- **`credentials`** : User authentication records. Primary key `credential_id` , unique index on `email` , hashed passwords using BCrypt ($2a$10$).
- **`refresh_tokens`** : Rotating refresh tokens for session renewal. Foreign reference to `credential_id` .
- **`email_verifications`** : OTP storage with 15-minute TTL expirations.

3.2 `aerobook_user_db`

- **`users`** : Master customer demographics. Primary key `user_id` , unique index on `email` .
- **`incident_reports`** : Passenger conduct and airport security reports. Columns: `id` , `user_id` , `user_name` , `user_email` , `reported_by` , `reason` , `severity` , `status` , `notes` .

3.3 `aerobook_flight_db`

- **`aircrafts`** : Commercial aircraft fleet inventory. Primary key `id` , unique index on `aircraft_code` .
- **`flights`** : Commercial flight schedules. Primary key `id` , unique index on `flight_number` , foreign key referencing `aircrafts.id` . Real-time `available_seats` tracking.

3.4 `aerobook_fare_db`

- **`fares`** : Cabin-class pricing tiers. Primary key `id` , unique foreign reference `flight_id` .
- **`coupons`** : Promotional discount codes. Primary key `id` , unique index on `code` . Fields include `usage_limit` , `usage_count` , and date bounds.

3.5 `aerobook_booking_db`

- **`bookings`**: Master reservation records. Primary key `booking_id`, unique index on `pnr`.
- **`passengers`**: Individual traveler details. Primary key `passenger_id`, foreign key referencing `bookings.booking_id`.

3.6 `aerobook_checkin_db`

- **`check_in`**: Boarding pass issuance records. Primary key `id`, unique index on `boarding_pass_number`, status transition from `CHECKED_IN` to `BOARDED`.

3.7 `aerobook_payment_db`

- **`payments`**: Financial transactions. Primary key `id`, unique index on `order_id`, tracking Razorpay `payment_id`, `signature`, and refund lifecycle.

3.8 `aerobook_notification_db`

- **`notifications`**: User alert inbox. Primary key `id`, index on `user_id`, storing title, message, status (`PENDING`, `SENT`, `READ`), and timestamps.

---

4. Master Data Seeding Strategy (`scripts/seed.sql`)

The platform includes an automated master database seeding script at `scripts/seed.sql` (39.6 KB, 429 lines).

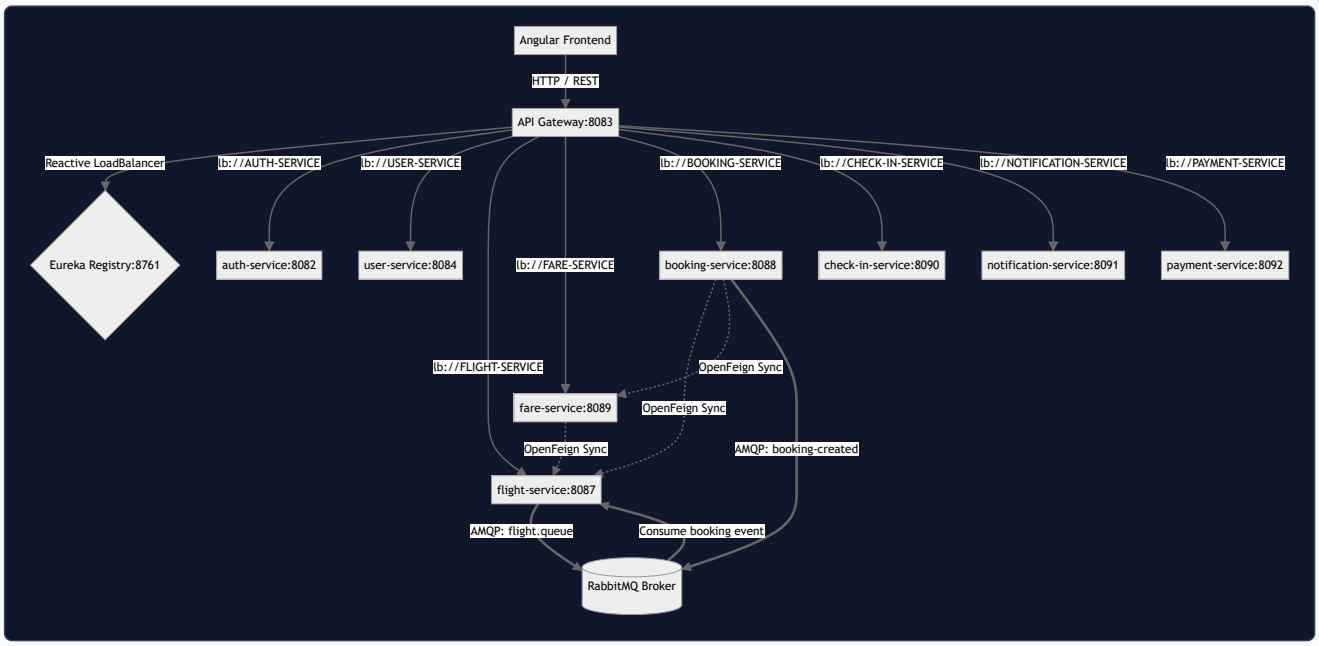
- Creates all 8 databases with `utf8mb4` encoding and collation `utf8mb4_unicode_ci`.
- Creates missing table schemas.
- Seeds 20 preconfigured user accounts (Admins, Airport Staff, and Passengers) with hashed BCrypt passwords.
- Seeds 15 commercial aircraft (Boeing 737, Airbus A320, Boeing 787).
- Seeds 30 scheduled commercial flights across major Indian airport hubs (DEL, BOM, BLR, CCU, HYD, MAA, GOI).
- Seeds tiered fares and active promotional coupons (`FLY500`, `AERO10`, `WELCOME20`).
- Seeds historical sample bookings, passenger manifests, check-ins, payments, notifications, and incident reports.

CHAPTER 9

Inter-Service Communication Architecture

1. Communication Paradigm Overview

AeroBook combines **synchronous HTTP REST (via Spring Cloud OpenFeign)** for query-response dependencies requiring immediate operational consistency, and **asynchronous AMQP (via RabbitMQ)** for state change propagation, eventual consistency, and cross-service event fanout.



2. Master Inter-Service Communication Matrix

Source Service	Target Service	Protocol / Transport	Pattern	Destination / Channel	Purpose
`api-gateway`	All Microservices	HTTP / REST	Request / Response	`lb://<SERVICE-NAME>`	Edge routing, JWT validation, header enrichment
`booking-service`	`flight-service`	HTTP / OpenFeign	Synchronous RPC	`GET /api/flights/{id}`	Validate flight status & available seat inventory
`booking-service`	`fare-service`	HTTP / OpenFeign	Synchronous RPC	`GET /api/fares/flight/{id}`	Retrieve cabin pricing for ticket cost calculation
`fare-service`	`flight-service`	HTTP / OpenFeign	Synchronous RPC	`GET /api/flights/{id}`	Verify flight existence before assigning fare pricing
`booking-service`	`flight-service`	AMQP / RabbitMQ	Asynchronous Event	Queue: `booking-created`	Atomically decrement available seats upon ticket confirmation
`flight-service`	Downstream Services	AMQP / RabbitMQ	Asynchronous Event	Queue: `flight.queue`	Broadcast schedule changes & seat release events
`auth-service`	`notification-service`	HTTP / REST	Synchronous RPC	`POST /api/notifications/email/*`	Dispatch registration OTPs and password reset emails
`booking-service`	`notification-service`	HTTP / REST	Synchronous RPC	`POST /api/notifications/email/booking`	Dispatch booking confirmation email with ticket PDF
`check-in-service`	`notification-service`	HTTP / REST	Synchronous RPC	`POST /api/notifications/email/checkin`	Dispatch boarding pass confirmation with PDF

3. Synchronous OpenFeign Implementation

3.1 FlightServiceClient (in `booking-service` & `fare-service`)

```
@FeignClient(name = "FLIGHT-SERVICE")
public interface FlightServiceClient {
    @GetMapping("/api/flights/{id}")
    FlightResponse getFlightById(@PathVariable("id") Long id);
}
```

- **Service Discovery Integration**: OpenFeign automatically queries Eureka for healthy instance coordinates under service ID `FLIGHT-SERVICE`.
- **Client-Side Load Balancing**: Spring Cloud LoadBalancer distributes concurrent requests across active instances using round-robin.

3.2 FareServiceClient (in `booking-service`)

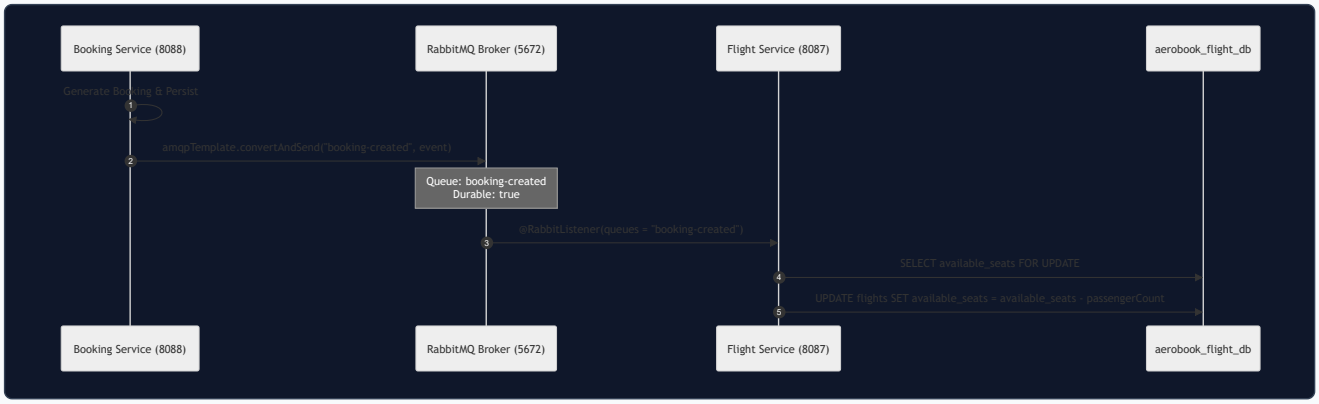
```
@FeignClient(name = "FARE-SERVICE")
public interface FareServiceClient {
    @GetMapping("/api/fares/flight/{flightId}")
    FareResponse getFareByFlightId(@PathVariable("flightId") Long flightId);
}
```

---

4. Asynchronous AMQP RabbitMQ Infrastructure

4.1 Topology Configuration

- **Broker Port**: `5672` (AMQP), `15672` (Management Dashboard)
- **Message Converter**: Jackson 2 JSON Message Converter (`Jackson2JsonMessageConverter`) ensures language-agnostic JSON payload serialization.



4.2 Resiliency & Fault Isolation

- **Durable Queues**: Queues are configured as `durable = true` to prevent message loss during broker restarts.
- **Decoupled Failures**: If `flight-service` undergoes temporary downtime or redeployment, booking transactions complete without failure; seat adjustment events remain queued until `flight-service` reconnects.

CHAPTER 10

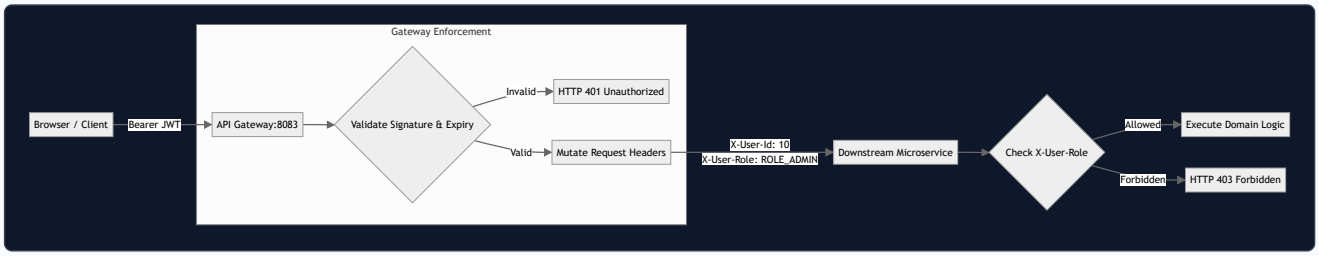
Platform Security & Authentication Architecture

1. Security Philosophy & Edge Enforcement

AeroBook implements a **defense-in-depth, perimeter-enforced security architecture**. The platform uses stateless **JSON Web Tokens (JWT)** for identity verification and access delegation.

Authentication is centralized at the **API Gateway** (`api-gateway:8083`):

1. **Edge JWT Verification**: Inbound requests with `Authorization: Bearer <token>` are cryptographically validated by the Gateway's reactive `JwtAuthenticationFilter` .
2. **Context Enrichment & Mutation**: Valid tokens have their claims extracted (`userId` , `email` , `role`). The Gateway mutates the downstream HTTP request, appending standard HTTP headers:
   - `X-User-Id` : Numeric ID of the authenticated user.
   - `X-User-Email` : Email address of the authenticated user.
   - `X-User-Role` : Assigned role (`ROLE_USER` , `ROLE_STAFF` , `ROLE_ADMIN`).
3. **Downstream Service Defense**: Microservices trust the gateway-propagated `X-User-*` headers, eliminating repeated asymmetric cryptography calculations while enforcing RBAC at the service layer.



2. JWT Token Specification & Cryptography

- **Library**: JJWT `0.12.7` (`jjwt-api` , `jjwt-impl` , `jjwt-jackson`).
- **Signature Algorithm**: HMAC-SHA256 (`HS256`).
- **Secret Key**: Defined via `jwt.secret` (minimum 256 bits, Base64/Hex encoded).
- **Token Lifespans**:
  - **Access Token TTL**: 24 hours (86,400,000 ms).
  - **Refresh Token TTL**: 7 days (604,800,000 ms).
- **Token Claims Payload**:

```
{
  "sub": "admin@aerobook.com",
  "userId": 10,
  "role": "ROLE_ADMIN",
  "iat": 1729238400,
  "exp": 1729324800
}
```

3. Role-Based Access Control (RBAC) Matrix

The platform classifies operations into three distinct personas:

- **`ROLE_USER` (Passenger)**: Can search flights, view schedules, book tickets, check in, download boarding passes, manage own profile, and cancel own bookings.
- **`ROLE_STAFF` (Airport Operations Staff)**: Can view passenger manifests, scan and verify boarding passes at departure gates (`/api/check-ins/verify`), search flights, view bookings, and file passenger incident reports.
- **`ROLE_ADMIN` (System Administrator)**: Full platform governance, including aircraft fleet registration, flight scheduling, cabin fare configuration, promo coupon creation, user role modifications, incident resolutions, and transaction auditing.

Feature / Domain	Endpoint Group	ROLE_USER	ROLE_STAFF	ROLE_ADMIN
Flight Search	`/api/flights/**` (GET)	Allowed	Allowed	Allowed
Fleet Management	`/api/admin/aircrafts/**`	Denied	View Only	Full CRUD
Flight Management	`/api/admin/flights/**`	Denied	View Only	Full CRUD

Feature / Domain	Endpoint Group	ROLE_USER	ROLE_STAFF	ROLE_ADMIN
Fare Configuration	`/api/fares/**`	View Only	View Only	Full CRUD
Coupon Management	`/api/fares/coupons/**`	Validate Only	Validate Only	Full CRUD
Booking Creation	`POST /api/bookings`	Allowed	Allowed	Allowed
Booking Oversight	`GET /api/bookings`	Denied	Allowed	Allowed
Self Check-In	`POST /api/check-ins`	Allowed	Allowed	Allowed
Gate Verification	`POST /api/check-ins/verify`	Denied	Allowed	Allowed
Manifest Audit	`GET /api/check-ins`	Denied	Allowed	Allowed
Incident Reports	`/api/v1/users/incidents/**`	Denied	File & View	Full CRUD
User Administration	`/api/v1/users/**`	Self Profile	View Manifest	Full CRUD
Role Modification	`PUT /api/auth/credentials/{id}/role`	Denied	Denied	Allowed
Payment Orders	`POST /api/payments/orders`	Allowed	Allowed	Allowed
Payment Refunds	`POST /api/payments/{id}/refund`	Denied	Allowed	Allowed

4. Administrative Bootstrapping & Setup Keys

To prevent unauthorized privilege escalation while allowing secure containerized bootstrapping, privileged registration endpoints enforce **Setup Keys**:

- `POST /api/auth/register-admin` : Requires header/body parameter matching `aerobook.setup-key.admin` (e.g. `ADMIN_SECRET_KEY_2026`).
- `POST /api/auth/register-staff` : Requires header/body parameter matching `aerobook.setup-key.staff` (e.g. `STAFF_SECRET_KEY_2026`).

5. Password Security & BCrypt Hashing

Passwords are never stored in plaintext. They are salted and hashed using `BCryptPasswordEncoder` with a cost factor (strength) of `10` :

- Standard BCrypt signature format: `$2a$10$...`
- Prevents rainbow table attacks, dictionary attacks, and timing leaks.

CHAPTER 11

Configuration Reference & Environment Management

1. Centralized Configuration Architecture

AeroBook supports both **decentralized service profiles** and **centralized Spring Cloud Config Server** (`config-server:8080`).

Each microservice imports the centralized configuration optionally via:

```
spring.config.import=optional:configserver:http://localhost:8080
```

This ensures that services can run standalone with sensible local defaults or inherit centralized parameters across `dev` , `docker` , and `prod` deployment profiles.

2. Master Environment Variables Inventory

Variable Name	Default / Example Value	Description	Services Consuming
`MYSQL_URL`	`jdbc:mysql://localhost:3306/<db_name>?createDatabaseIfNotExist=true`	JDBC connection string	All 8 persistent services
`MYSQL_USERNAME`	`root`	Database master user	All 8 persistent services
`MYSQL_PASSWORD`	`[SECURE_CREDENTIAL]`	Database user password	All 8 persistent services
`EUREKA_URL`	`http://localhost:8761/eureka/`	Eureka Service Registry endpoint	All 10 client services
`RABBITMQ_HOST`	`localhost`	AMQP Message Broker host	`booking-service` , `flight-service`
`RABBITMQ_PORT`	`5672`	AMQP Message Broker port	`booking-service` , `flight-service`
`RABBITMQ_USERNAME`	`guest`	RabbitMQ user account	`booking-service` , `flight-service`
`RABBITMQ_PASSWORD`	`[SECURE_CREDENTIAL]`	RabbitMQ user password	`booking-service` , `flight-service`
`JWT_SECRET`	`[MINIMUM_32_CHAR_SECRET_KEY]`	HMAC-SHA256 signing key	`api-gateway` , `auth-service` , `booking-service`
`JWT_EXPIRATION`	`3600000` (1 hour)	JWT Access Token TTL in milliseconds	`auth-service`
`JWT_REFRESH_EXPIRATION`	`604800000` (7 days)	Refresh Token TTL in milliseconds	`auth-service`
`ADMIN_REGISTRATION_KEY`	`[SECURE_ADMIN_KEY]`	Setup key required for admin signups	`auth-service`
`STAFF_REGISTRATION_KEY`	`[SECURE_STAFF_KEY]`	Setup key required for staff signups	`auth-service`
`RAZORPAY_KEY_ID`	`rzp_test_...`	Public client identifier for Razorpay	`payment-service` , `frontend`
`RAZORPAY_KEY_SECRET`	`[SECURE_KEY_SECRET]`	Private server secret for Razorpay	`payment-service`
`RAZORPAY_WEBHOOK_SECRET`	`[SECURE_WEBHOOK_SECRET]`	Secret for verifying Razorpay webhooks	`payment-service`
`SPRING_MAIL_HOST`	`smtp.gmail.com`	Outbound SMTP host	`notification-service`
`SPRING_MAIL_PORT`	`587`	SMTP TLS port	`notification-service`
`SPRING_MAIL_USERNAME`	`operations@aerobook.com`	SMTP authenticated account	`notification-service`
`SPRING_MAIL_PASSWORD`	`[SECURE_APP_PASSWORD]`	App-specific mail password	`notification-service`
`AEROBOOK_MAIL_ENABLED`	`true`	Toggle outbound mail dispatch	`notification-service`

3. Configuration Profiles

3.1 Local Development Profile (`default`)

- Points to `localhost` databases, local Eureka (`8761`), and local RabbitMQ.
- DDL schema auto-generation: `spring.jpa.hibernate.ddl-auto=update` .
- SQL statement logging enabled in debug mode.

3.2 Docker Profile (`docker`)

- Activated via `-Dspring.profiles.active=docker` or `SPRING_PROFILES_ACTIVE=docker` .
- Targets Docker internal hostnames:
  - Database: `mysql:3306`
  - Eureka: `http://service-discovery:8761/eureka/`
  - RabbitMQ: `rabbitmq:5672`
  - Config Server: `http://config-server:8080`

3.3 Production Profile (`prod`)

- DDL auto-generation set to `validate` (schema mutations managed strictly via migration scripts).
- Sensitive environment variables supplied via container secrets or Kubernetes Secrets.
- Actuator endpoints restricted to internal cluster subnets.

CHAPTER 12

Local Development & Quick Start Guide

1. System Prerequisites

Before running the AeroBook microservices ecosystem locally, ensure the following runtimes and tools are installed:

- **Java Development Kit (JDK)**: OpenJDK 21 LTS (`java -version`)
- **Build Tool**: Apache Maven 3.8+ (`mvn -version`) or Maven Wrapper (`./mvnw`)
- **Node.js & npm**: Node.js v18.19+ or v20/v22 LTS, npm 10+ (`node -v` , `npm -v`)
- **Angular CLI**: Angular CLI 18.x (`ng version`)
- **Database Engine**: MySQL Server 8.0+ running on port `3306`
- **Message Broker**: RabbitMQ Server running on port `5672` (Erlang OTP installed)
- **PowerShell / Bash**: PowerShell 5.1+ (Windows) or GNU Bash (Linux / macOS)

---

2. Step-by-Step Setup Sequence

Step 1: Initialize Database Schemas and Seed Data

Execute the seed script to create all 8 microservice databases and populate them with initial mock aircraft, flight schedules, fares, and user credentials:

On Windows (PowerShell):

```
.\scripts\seed.ps1
```

On Linux / macOS (Bash):

```
./scripts/seed.sh
```

Manual MySQL Seed:

```
mysql -u root -p < scripts/seed.sql
```

Step 2: Build All Backend Microservices

Run the automated build orchestrator to compile and package all 11 Java services:

```
.\scripts\build-all.ps1
```

Or manually run Maven in any service directory:

```
cd auth-service
mvn clean package -DskipTests
```

Step 3: Install Frontend Dependencies

```
cd frontend
npm install
```

Step 4: Launch the Microservices Ecosystem

AeroBook includes a smart orchestration script that handles phased startup sequencing (Eureka Server first, then Config Server, then downstream business microservices):

Start all services in background:

```
.\scripts\run-services.ps1
```

Start all services in visible individual terminal windows:

```
.\scripts\run-services.ps1 -Window
```

Start all services including the Angular Frontend:

```
.\scripts\run-services.ps1 -Frontend
```

Check running status of all services:

```
.\scripts\run-services.ps1 status
```

Stop all running services:

```
.\scripts\run-services.ps1 stop
```

3. Verify Operational Health

Execute the platform smoke test to verify all service health probes directly and through the API Gateway:

```
.\scripts\smoke-check.ps1
```

Direct Service Ports & Dashboards

Service / Dashboard	URL	Default Port
Angular Frontend	`http://localhost:4200`	4200
API Gateway	`http://localhost:8083`	8083
Eureka Service Registry	`http://localhost:8761`	8761
Config Server	`http://localhost:8080`	8080
Auth Service	`http://localhost:8082`	8082
User Service	`http://localhost:8084`	8084
Flight Service	`http://localhost:8087`	8087
Booking Service	`http://localhost:8088`	8088
Fare Service	`http://localhost:8089`	8089
Check-In Service	`http://localhost:8090`	8090
Notification Service	`http://localhost:8091`	8091
Payment Service	`http://localhost:8092`	8092
RabbitMQ Management	`http://localhost:15672`	15672

4. Default Seeded User Credentials

Persona	Email	Password	Access Level
System Administrator	`admin@aerobook.com`	`Admin@123`	Full access to Admin Dashboard (`/admin`)
Airport Staff	`staff@aerobook.com`	`Staff@123`	Staff Dashboard (`/staff`) & Gate Scanner
Passenger 1	`priya@gmail.com`	`Passenger@123`	Customer Dashboard (`/customer`), Bookings
Passenger 2	`asha.khan@example.com`	`Password123!`	Customer Dashboard (`/customer`)
Passenger 3	`sohel@aerobook.com`	`Admin@123`	Customer & Admin privileges

CHAPTER 13

Deployment & Containerization Architecture

1. Containerization Strategy

AeroBook is architected for containerized deployment across local development, staging, and cloud container orchestration platforms (Docker Swarm, Kubernetes, Amazon ECS).

Each component follows modern container packaging standards:

- **Java 21 Spring Boot Microservices**: Packaged using multi-stage Docker builds based on `eclipse-temurin:21-jre-alpine` for minimal attack surface and sub-200MB image footprints.
- **Angular 18 Frontend**: Built using Node.js `20-alpine` and served via high-performance `nginx:alpine` with HTML5 pushState routing rules and gzip/brotli compression.

2. Standard Multi-Stage Dockerfile Patterns

2.1 Backend Spring Boot Services

```
# Stage 1: Build & Package
FROM maven:3.9.8-eclipse-temurin-21-alpine AS builder
WORKDIR /workspace
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

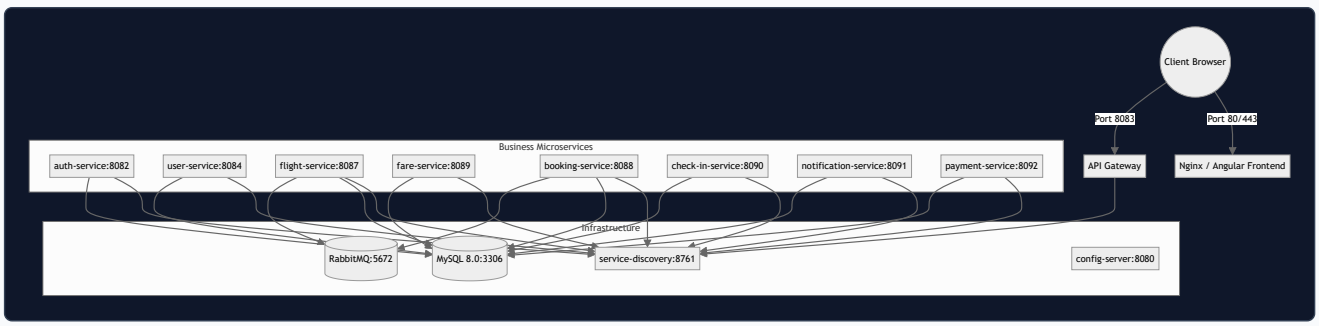
# Stage 2: Minimal Distroless / JRE Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=builder /workspace/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:+UseG1GC", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

2.2 Angular 18 Frontend

```
# Stage 1: Build Single Page Application
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve via Nginx
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist/frontend/browser /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3. Production Docker Compose Architecture



4. Kubernetes Deployment Architecture (Production Target)

For enterprise production scale, AeroBook maps directly into standard Kubernetes constructs:

```
k8s/
├── namespaces/
│   └── aerobook.yaml               # Dedicated isolated namespace
├── configmaps/
│   └── platform-config.yaml        # Common environment parameters
├── secrets/
│   └── platform-secrets.yaml       # Base64 encoded DB passwords & JWT secrets
├── deployments/
│   ├── eureka-deployment.yaml      # 2 replicas with pod anti-affinity
│   ├── gateway-deployment.yaml     # 3 replicas with HorizontalPodAutoscaler
│   └── microservices-*.yaml        # Individual stateless microservice deployments
└── ingress/
    └── aerobook-ingress.yaml       # Nginx Ingress Controller with Let's Encrypt TLS
```

Readiness & Liveness Probes

Every Spring Boot deployment leverages Spring Actuator endpoints:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 45
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5
```

---

5. Graceful Shutdown & Zero Downtime

All Spring Boot services have graceful shutdown enabled:

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

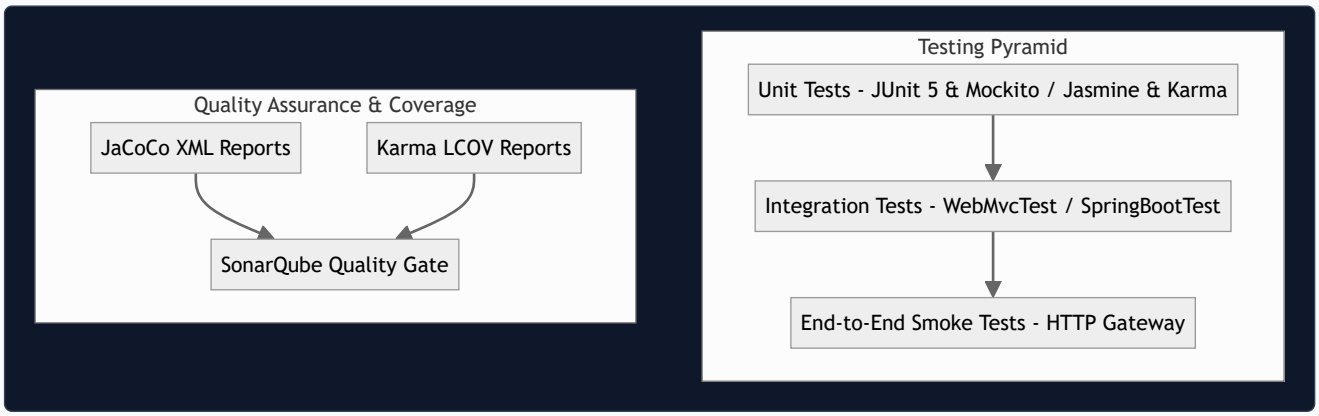
When SIGTERM is received during rolling updates, embedded Tomcat/Netty halts new inbound connections while permitting in-flight HTTP requests and AMQP messages up to 30 seconds to complete.

CHAPTER 14

Testing & Quality Assurance Architecture

1. Testing Strategy Overview

AeroBook adopts a comprehensive testing pyramid encompassing unit testing, mock-isolated service layer testing, slice controller testing (`@WebMvcTest`), end-to-end smoke tests, and automated static code analysis via **SonarQube**.



2. Backend Unit & Integration Testing

- **Framework**: JUnit 5 (Jupiter) & Mockito.
- **Assertion Engine**: AssertJ (`org.assertj.core.api.Assertions`).
- **Spring Testing Support**: `spring-boot-starter-test` .

2.1 Service Layer Testing Pattern

Service tests utilize Mockito to mock repository layers and external Feign clients, verifying business validation, domain rules, and exception throwing in total isolation from the database:

```
@ExtendWith(MockitoExtension.class)
class BookingServiceImplTest {
    @Mock private BookingRepository bookingRepository;
    @Mock private FlightServiceClient flightClient;
    @Mock private FareServiceClient fareClient;
    @Mock private RabbitTemplate rabbitTemplate;
    @InjectMocks private BookingServiceImpl bookingService;

    @Test
    void createBooking_Successful_ReturnsConfirmedResponse() {
        // Mock seat availability & fare calculations
        // Assert PNR generation and status == CONFIRMED
    }
}
```

2.2 Controller Slice Testing Pattern

Controllers are tested using `@WebMvcTest` with Spring Security mock contexts to verify HTTP status codes, request body validations (`@Valid`), and serialization behaviors:

```
@WebMvcTest(FlightSearchController.class)
class FlightSearchControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private FlightSearchService flightSearchService;

    @Test
    void searchFlights_ValidQuery_ReturnsOk() throws Exception {
        mockMvc.perform(get("/api/flights/search")
                .param("source", "DEL")
                .param("destination", "BOM"))
                .andExpect(status().isOk());
    }
}
```

3. Frontend Component & Service Testing

- **Test Runner**: Karma (`karma.conf.js`).
- **Framework**: Jasmine 5.x.
- **Coverage Tool**: `karma-coverage` generating HTML and LCOV reports at `frontend/coverage/lcov.info` .

Test Execution Commands

```
cd frontend
# Run tests headlessly for CI/CD
npm test -- --watch=false --browsers=ChromeHeadless --code-coverage
```

---

4. Code Coverage Reporting (JaCoCo)

All 11 backend microservices are configured with the **JaCoCo Maven Plugin** (`jacoco-maven-plugin:0.8.12`):

- Automatically binds to the Maven `test` and `prepare-agent` lifecycles.
- Generates standard XML coverage reports at `target/site/jacoco/jacoco.xml` .
- Standard exclusions: Generated configuration classes, Lombok/DTO boilerplate, and entity POJOs.

Running Backend Coverage

```
mvn clean test jacoco:report
```

---

5. SonarQube Code Quality & Security Analysis

SonarQube is integrated across all microservices and the Angular frontend.

5.1 SonarQube Metrics & Quality Gates

- **Bugs & Vulnerabilities**: 0 Blocker, 0 Critical.
- **Security Hotspots**: 100% reviewed.
- **Code Smells & Technical Debt**: Debt ratio under 5%.
- **Duplicated Lines**: Less than 3.0% duplication.
- **Test Coverage**: Tracked via ingested JaCoCo XML and LCOV reports.

5.2 Executing SonarQube Scans

```
# Analyze all Java services
mvn sonar:sonar -Dsonar.host.url=http://localhost:9000 -Dsonar.token=[TOKEN]

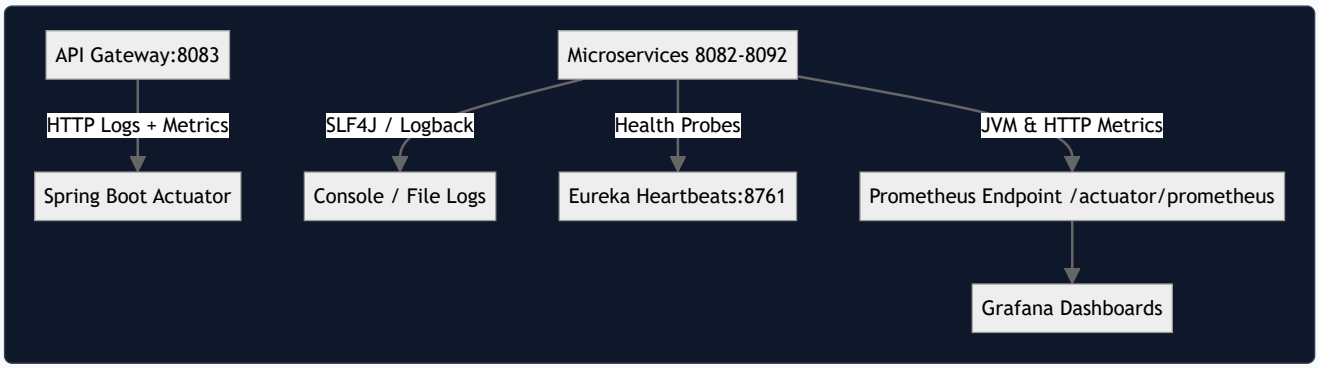
# Analyze Angular Frontend
cd frontend
npx sonar-scanner \
  -Dsonar.projectKey=aerobook-frontend \
  -Dsonar.sources=src \
  -Dsonar.tests=src \
  -Dsonar.test.inclusions=**/*.spec.ts \
  -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info \
  -Dsonar.host.url=http://localhost:9000 \
  -Dsonar.token=[TOKEN]
```

CHAPTER 15

Observability, Monitoring & Logging Architecture

1. Observability Strategy Overview

AeroBook adopts a distributed observability architecture structured around three key pillars: **Centralized Logging**, **Health & Readiness Probes**, and **Application Performance Metrics**.



2. Distributed Logging Architecture

2.1 SLF4J & Logback Configuration

All microservices implement SLF4J backed by Logback, utilizing a standardized ANSI colorized logging pattern:

```
logging.level.root=INFO
logging.level.com.aerobook=DEBUG
logging.level.org.springframework.cloud.gateway=INFO
logging.pattern.console=%clr(%d{yyyy-MM-dd HH:mm:ss.SSS}){faint} %clr(%5p) %clr(${PID:- }){magenta} --- [%15.15t] %clr(%-40.40logger{39}){cyan} : %m%n
```

2.2 Distributed Correlation IDs

To trace user transactions across the microservices ecosystem, the API Gateway intercepts incoming requests and attaches an `X-Correlation-Id` header (using a unique UUID) if not already provided.

- Downstream services log the correlation ID in their Mapped Diagnostic Context (MDC).
- Allows engineers to trace an entire distributed flow (e.g. `Client → Gateway → Booking Service → Flight Service → RabbitMQ`) using a single correlation search key.

3. Health Checks & Spring Boot Actuator

Every microservice exposes operational health and telemetry under the `/actuator` management context.

3.1 Actuator Endpoints Directory

Endpoint	Path	Purpose
Health Probe	`/actuator/health`	Comprehensive health check evaluating database connectivity, disk space, and AMQP broker connections.
Liveness Probe	`/actuator/health/liveness`	Kubernetes liveness probe indicating if the JVM process is alive and responsive.
Readiness Probe	`/actuator/health/readiness`	Kubernetes readiness probe indicating if the service has completed initialization and can accept traffic.
Service Metrics	`/actuator/metrics`	Real-time JVM memory heap usage, garbage collection latency, thread pools, and HTTP request metrics.
Application Info	`/actuator/info`	Application name, build version, and Spring Boot version metadata.

3.2 Actuator Configuration

```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
management.endpoint.health.show-details=always
management.endpoint.health.probes.enabled=true
```

4. Eureka Service Discovery Telemetry

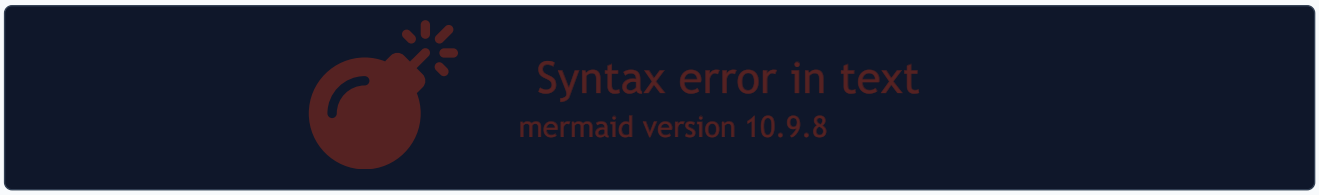
- **Heartbeat Interval**: Client microservices send heartbeats every 30 seconds (`eureka.instance.lease-renewal-interval-in-seconds=30`).
- **Lease Expiration**: Registry marks unrenewed instances as down after 90 seconds.
- **Self-Preservation Mode**: If network partitions cause more than 15% of heartbeats to drop suddenly, Eureka activates self-preservation mode to prevent mass deregistration of healthy instances.
- **Admin Dashboard**: Accessible at `http://localhost:8761` displaying active instances, IP addresses, up/down statuses, and JVM memory.

CHAPTER 16

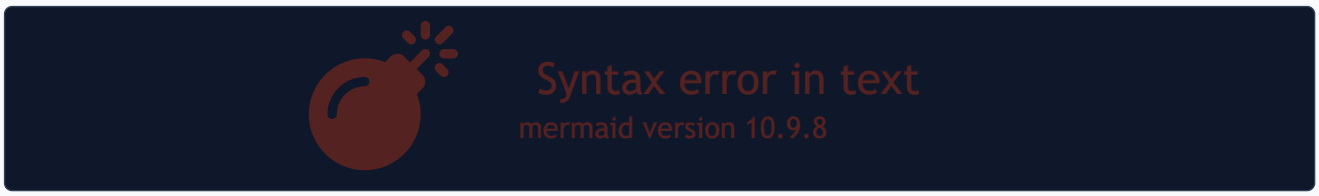
Core Business Workflows & Sequence Flows

This document details the primary end-to-end business processes in the AeroBook aviation platform, accompanied by comprehensive Mermaid sequence diagrams.

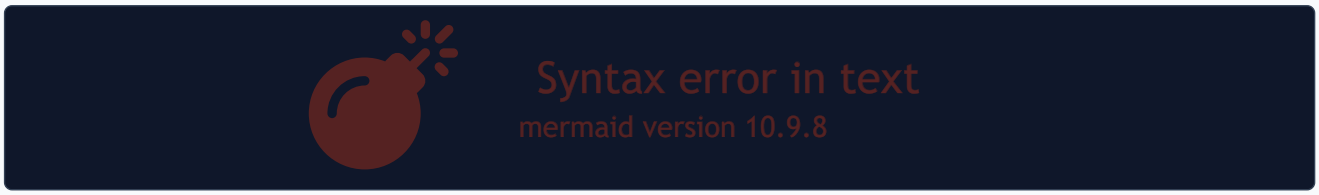
Workflow 1: User Registration, OTP Verification & Authentication



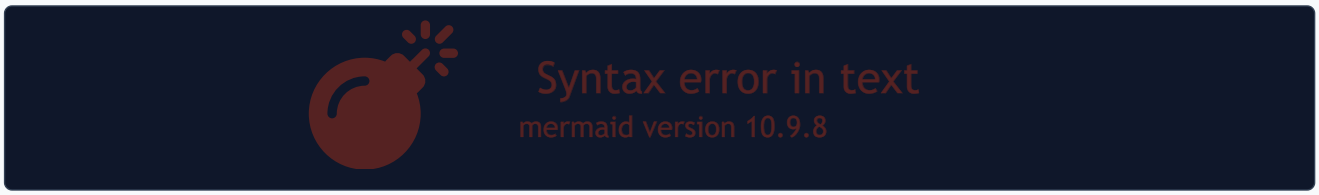
Workflow 2: Flight Search & Real-Time Availability Lookup



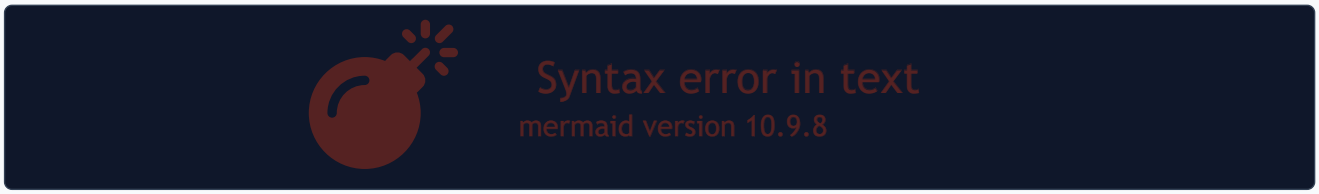
Workflow 3: Booking Reservation & Seat Allocation



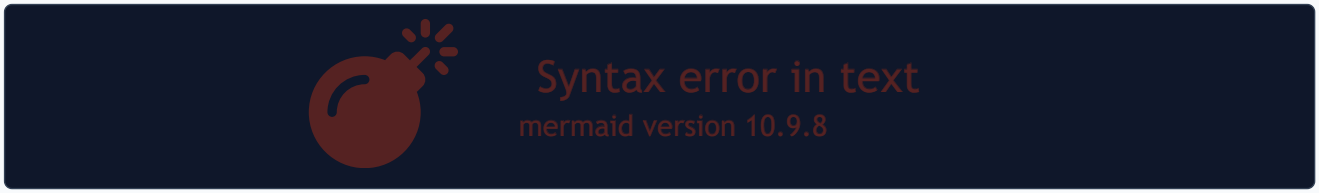
Workflow 4: Razorpay Payment Processing & Verification



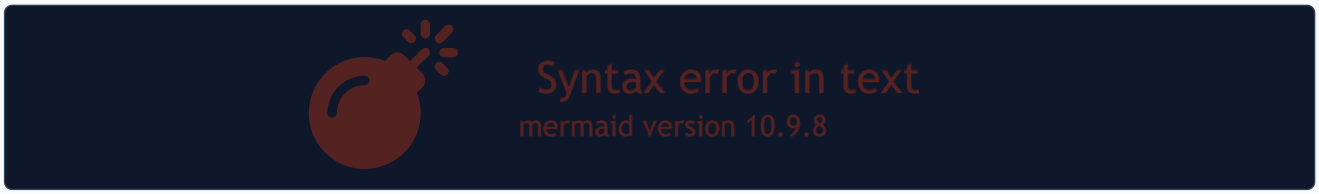
Workflow 5: Passenger Check-In & Boarding Pass Issuance



Workflow 6: Airport Departure Gate Verification



Workflow 7: Booking Cancellation & Automated Refund



CHAPTER 17

Operational Troubleshooting & Diagnostics Runbook

This guide details common operational failure modes encountered across the AeroBook microservices ecosystem, root cause analyses, and step-by-step remediation procedures.

---

1. Port Binding Conflicts & Stale Processes

Symptom

A microservice fails during startup with `Web server failed to start. Port 808X was already in use.`

Root Cause

A previously running instance or crashed JVM retained the TCP socket binding.

Remediation

Windows (PowerShell):

```
# Identify process ID holding the port (e.g. 8083)
Get-NetTCPConnection -LocalPort 8083 -ErrorAction SilentlyContinue | Select-Object OwningProcess

# Terminate the process
Stop-Process -Id <PID> -Force
```

Linux / macOS:

```
lsof -i :8083
kill -9 <PID>
```

---

2. Eureka Service Registration Latency & HTTP 503 Errors

Symptom

Immediately after launching services, frontend requests through `http://localhost:8083/api/...` return `HTTP 503 Service Unavailable` or `Unable to find instance for <SERVICE-NAME>`.

Root Cause

Netflix Eureka uses a 30-second heartbeat cycle. When a microservice boots up, it takes between 15 to 30 seconds for the Eureka server to register the lease, and another 15 to 30 seconds for the API Gateway's reactive discovery locator cache to synchronize.

Remediation

1. Wait 30 seconds after launching all microservices.
2. Verify the service appears as `UP` in the Eureka Web Console at `http://localhost:8761`.
3. Check the Gateway health registry at `http://localhost:8083/api/gateway/services`.
4. Run the health probe:

```
.\scripts\smoke-check.ps1
```

---

3. Database Connectivity & Authentication Failures

Symptom

Microservice logs display: `java.sql.SQLNonTransientConnectionException: Could not create connection to database server` or `Access denied for user 'root'@'localhost'`.

Root Cause

1. Local MySQL Server service is stopped.
2. The password in `application.properties` does not match your local MySQL root password.
3. The required database schema has not been initialized.

Remediation

1. Check MySQL service status:

```
Get-Service -Name MySQL*
Start-Service -Name MySQL80  # or your installed service name
```

2. Verify database schemas exist by re-running the seed script:

```
.\scripts\seed.ps1
```

3. Override credentials via environment variables without editing code:

```
$env:MYSQL_PASSWORD="YourActualPassword"
```

---

4. RabbitMQ Broker Connection Failures

Symptom

`booking-service` or `flight-service` fails with: `org.springframework.amqp.AmqpConnectException: java.net.ConnectException: Connection refused: no further information`.

Root Cause

RabbitMQ server is not running or the AMQP default port `5672` is blocked.

Remediation

1. Verify RabbitMQ Windows Service is running:

```
Get-Service -Name RabbitMQ
Start-Service -Name RabbitMQ
```

2. Check RabbitMQ management console at `http://localhost:15672` (default credentials: `guest` / `guest`).

---

5. Frontend CORS Errors

Symptom

Browser console logs `Access to fetch at 'http://localhost:808X/...' has been blocked by CORS policy`.

Root Cause

The frontend attempted to connect directly to an individual microservice port (e.g. `8087` or `8088`) rather than routing through the centralized API Gateway on port `8083`.

Remediation

1. Verify all API requests in the Angular client target `http://localhost:8083`.
2. Ensure the API Gateway `globalcors` configuration is loaded.

---

6. JWT Authentication & Token Expiration

Symptom

Protected endpoints return `HTTP 401 Unauthorized` with message `JWT expired` or `SignatureException`.

Root Cause

Access tokens expire after 24 hours. Alternatively, if `jwt.secret` differs between `auth-service` and `api-gateway`, signature verification fails.

Remediation

1. Ensure `jwt.secret` matches in both `auth-service/src/main/resources/application.properties` and `api-gateway/src/main/resources/application.properties`.
2. Log out and log back in, or trigger the refresh token endpoint:

```
POST http://localhost:8083/api/auth/refresh-token
Content-Type: application/json

{ "refreshToken": "..." }
```

---

7. Role Synchronization & Admin Dashboard Redirection

Symptom

After logging in with an Admin account (`admin@aerobook.com`), the user is not automatically redirected to `/admin` or receives access denied.

Root Cause

Role claims in the JWT token or database may contain `ROLE_ADMIN` while frontend guard checks expected `ADMIN` (or vice versa), or local storage was not cleared from a previous unprivileged session.

Remediation

1. `AuthService` in the frontend normalizes roles by stripping the `ROLE_` prefix or checking both (`ROLE_ADMIN` and `ADMIN`).
2. Clear browser `localStorage` (`localStorage.clear()`) and re-login.
3. Verify that the database `aerobook_auth_db.credentials` table has `role = 'ROLE_ADMIN'` .

---

8. Razorpay Signature Verification Discrepancies

Symptom

Payment verification returns `HTTP 400 Bad Request` with `Invalid Payment Signature` .

Root Cause

The `razorpay.key.secret` configured in `payment-service` does not match the secret of the Razorpay Key ID used by the frontend checkout script.

Remediation

1. Verify that `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` in `payment-service/src/main/resources/application.properties` belong to the same active Razorpay test account pair.

CHAPTER 18

Domain & Technical Glossary

This glossary defines standard domain terms from commercial aviation and technical terms from cloud-native microservices architecture used throughout the AeroBook platform.

1. Aviation & Travel Domain Terminology

Term	Definition
PNR (Passenger Name Record)	A unique 6-character alphanumeric identifier generated by the reservation system (e.g., `AERO9X`) that acts as the master digital record for a passenger's itinerary, ticketing, and seat assignments.
IATA Airport Code	A three-letter geocode designated by the International Air Transport Association identifying an airport (e.g., `DEL` for Delhi Indira Gandhi International, `BOM` for Mumbai Chhatrapati Shivaji Maharaj).
Boarding Pass	A physical or digital document issued during airport check-in granting a passenger permission to board an aircraft, containing passenger name, flight number, seat assignment, gate, and barcode identifier (`BP—<bookingId>-<seatNumber>`).
Cabin Class	A division of airline passenger seating with distinct pricing, baggage allowances, and amenities. AeroBook supports `ECONOMY`, `BUSINESS`, and `FIRST_CLASS`.
Base Fare	The foundational price of a flight ticket before the addition of mandatory aviation taxes, fuel surcharges, airport development fees, and promo discounts.
Flight Status	Operational state of a scheduled commercial flight: `SCHEDULED`, `ON_TIME`, `DELAYED`, `BOARDING`, `DEPARTED`, `ARRIVED`, or `CANCELLED`.
Aircraft Registration	Unique alphanumeric code identifying a civil aircraft (e.g., `VT-EXA`), analogous to an automobile license plate. In India, aircraft registrations begin with the prefix `VT-`.
Manifest	Complete passenger and crew roster on a scheduled flight used for TSA, immigration, and gate boarding audits.
Gate Verification	The process by airport operations staff scanning the passenger boarding pass barcode at the departure gate, changing their status from `CHECKED_IN` to `BOARDED`.

2. Cloud & Microservices Architecture Terminology

Term	Definition
API Gateway	A single perimeter reverse-proxy and routing service (`api-gateway:8083`) that acts as the single point of entry for client applications, handling SSL termination, JWT authentication, rate limiting, and request routing to downstream microservices.
Service Discovery (Eureka)	A dynamic registry (`service-discovery:8761`) where microservices register their IP addresses, ports, and operational health, enabling decoupled inter-service resolution without hardcoded network URLs.
Client-Side Load Balancing	A mechanism (via Spring Cloud LoadBalancer) where the calling microservice queries the service registry and distributes outbound HTTP requests across available healthy instances using algorithms like Round Robin.
Spring Cloud OpenFeign	A declarative REST client library allowing Java microservices to invoke remote HTTP endpoints using annotated Java interfaces rather than manual `RestTemplate` calls.
Database-Per-Service	An enterprise microservices pattern where each service strictly owns its database schema, preventing tight coupling and ensuring that data mutations can only occur via defined service APIs or event queues.
AMQP (Advanced Message Queuing Protocol)	An open standard application layer protocol for message-oriented middleware used by RabbitMQ to ensure reliable, asynchronous, decoupled message delivery between services.
JWT (JSON Web Token)	A compact, URL-safe standard (RFC 7519) representing claims transferred between parties, signed using HMAC-SHA256 to ensure authenticity and integrity without server-side session state.
RBAC (Role-Based Access Control)	An access governance approach that restricts system operations based on assigned user roles: `ROLE_USER`, `ROLE_STAFF`, and `ROLE_ADMIN`.
Angular Signals	A reactive primitive introduced in modern Angular that tracks state dependencies and performs fine-grained, surgical DOM updates without triggering full component tree change detection cycles.
JaCoCo (Java Code Coverage)	A bytecode-level code coverage library for Java that computes branch and line coverage during test execution, exporting structured reports for SonarQube ingestion.

CHAPTER 19

Architecture Decision Records (ADR)

This document captures the key architectural decisions, rationale, trade-offs, and alternatives considered during the design and evolution of the AeroBook platform.

ADR 01: Microservices Architecture over Monolithic Architecture

- **Status**: Accepted
- **Context**: An airline booking and operations platform encompasses multiple distinct business boundaries with divergent scaling, security, and operational requirements (e.g., high-volume read-heavy flight searches vs. high-concurrency transactional booking reservations and low-latency gate boarding verifications).
- **Decision**: Decompose the application into 11 independently deployable microservices organized around domain boundaries: Service Discovery, Config Server, API Gateway, Auth, User, Flight, Fare, Booking, Check-In, Notification, and Payment.
- **Consequences**:
  - *Positive*: Independent scaling, domain isolation, localized failure domains, autonomous service deployments.
  - *Trade-offs*: Increased operational complexity, distributed transactions requiring eventual consistency, inter-service network latency.

ADR 02: Database-per-Service Pattern over Shared Database Schema

- **Status**: Accepted
- **Context**: In monolithic enterprise databases, multiple subsystems execute cross-table joins, tightly coupling database schemas and preventing individual teams or services from modifying data models without risk of system-wide breakage.
- **Decision**: Mandate strict Database-per-Service isolation across 8 private MySQL schemas (`aerobook_auth_db` , `aerobook_user_db` , `aerobook_flight_db` , `aerobook_fare_db` , `aerobook_booking_db` , `aerobook_checkin_db` , `aerobook_notification_db` , `aerobook_payment_db`). Microservices are prohibited from querying external databases directly.
- **Consequences**:
  - *Positive*: Complete schema autonomy, independent database performance tuning, zero cross-domain table locking.
  - *Trade-offs*: Elimination of relational foreign key constraints across services; cross-service queries must be orchestrated via APIs or messaging.

ADR 03: Reactive Non-Blocking API Gateway (Netty & Spring Cloud Gateway)

- **Status**: Accepted
- **Context**: The API Gateway serves as the single perimeter ingress for all platform traffic. A thread-per-request blocking gateway model (e.g., Spring MVC / Zuul 1) can suffer thread pool exhaustion under heavy concurrent request spikes.
- **Decision**: Adopt Spring Cloud Gateway built on Spring WebFlux, Project Reactor, and Netty.
- **Consequences**:
  - *Positive*: Asynchronous non-blocking I/O capable of handling tens of thousands of concurrent client connections with minimal memory and CPU overhead.
  - *Trade-offs*: Reactive programming paradigms require careful filter chain structuring and non-blocking filter implementations.

ADR 04: Hybrid Synchronous (OpenFeign) & Asynchronous (RabbitMQ) Messaging

- **Status**: Accepted
- **Context**: Certain business actions require immediate consistent answers (e.g., Booking Service must verify flight existence and seat counts before accepting payment), while other operations can be processed asynchronously without blocking the user (e.g., flight seat count deduction, e-ticket PDF generation, email dispatch).
- **Decision**: Use Spring Cloud OpenFeign with Eureka client-side load balancing for synchronous query-response flows, and RabbitMQ with durable AMQP queues for asynchronous state propagation and background notifications.
- **Consequences**:
  - *Positive*: Optimal user response times, fault isolation (booking succeeds even if notifications are delayed), guaranteed message delivery via durable queues.
  - *Trade-offs*: Requires maintaining both an HTTP service discovery mesh and an AMQP broker cluster.

ADR 05: Perimeter JWT Authentication & Downstream Header Mutation

- **Status**: Accepted
- **Context**: Re-evaluating cryptographic JWT signatures and hitting database caches on every internal microservice hop introduces significant CPU overhead and latency.
- **Decision**: Centralize JWT signature verification and role parsing exclusively at the API Gateway. The Gateway extracts claims and injects trusted HTTP headers (`X-User-Id` , `X-User-Role` , `X-User-Email`) into downstream requests.
- **Consequences**:
  - *Positive*: High internal throughput, downstream services remain lightweight and decoupled from token parsing logic.
  - *Trade-offs*: Internal service ports must be secured from direct external ingress via network firewalls or container virtual networks.

ADR 06: Angular 18 Standalone Components & Signals over NgModules

- **Status**: Accepted
- **Context**: Traditional Angular applications suffer from NgModule boilerplate, verbose RxJS boilerplate for simple reactive state, and performance overhead from Zone.js dirty checking.
- **Decision**: Build the frontend Single Page Application using Angular 18 with 100% Standalone Components and native Angular Signals (`signal()` , `computed()`) for component state management.
- **Consequences**:
  - *Positive*: Tree-shakable bundle sizes, clear component dependency graphs, instantaneous fine-grained DOM updates without zone-wide re-renders.
  - *Trade-offs*: Developers must adhere strictly to Signal-based data patterns rather than legacy two-way bindings.

CHAPTER 20

Architecture Evolution & Engineering Recommendations

This document outlines prioritized, actionable engineering recommendations to advance the AeroBook microservices platform from a high-functioning development state to an enterprise-grade, high-availability production ecosystem.

---

1. High Priority (Near-Term Production Readiness)

1.1 Database Schema Migrations (Flyway / Liquibase)

- **Current State**: Microservices rely on Hibernate DDL auto-generation (`spring.jpa.hibernate.ddl-auto=update`).
- **Risk**: Automatic schema generation can cause accidental column mutations, cannot roll back failed schema alterations, and is unsafe for zero-downtime database upgrades.
- **Recommendation**: Integrate **Flyway** across all 8 persistent services. Set `spring.jpa.hibernate.ddl-auto=validate` and manage schema changes via versioned SQL scripts (`V1__initial_schema.sql` , `V2__add_index.sql`).

1.2 Circuit Breakers & Resilience4j

- **Current State**: OpenFeign clients (`FlightServiceClient` , `FareServiceClient`) invoke downstream HTTP services directly.
- **Risk**: Cascading service failure if a downstream microservice slows down or becomes unavailable during high load.
- **Recommendation**: Equip all OpenFeign client interfaces with **Resilience4j Circuit Breakers**, TimeLimiters, and fallbacks to gracefully handle downstream service timeouts and return cached flight schedules or friendly error states.

1.3 Distributed Caching with Redis

- **Current State**: Flight search queries (`GET /api/flights/search`) execute SQL table queries on `aerobook_flight_db` on every user keystroke and search submission.
- **Risk**: High database CPU utilization during flight search peak hours.
- **Recommendation**: Introduce a **Redis cache cluster** for flight route schedules, seat availability read models, and active coupon codes with a 60-second time-to-live (TTL), invalidated via AMQP events upon flight modification.

1.4 API Gateway Rate Limiting & Throttling

- **Current State**: Gateway routes requests without request rate limitations.
- **Risk**: Vulnerable to brute-force credential stuffing attacks on `/api/auth/login` and denial-of-service on search endpoints.
- **Recommendation**: Enable Spring Cloud Gateway's built-in **Redis RateLimiter** using the Token Bucket algorithm, enforcing limits such as 10 requests/sec for anonymous search and 5 requests/min for login attempts.

---

2. Medium Priority (Operational Maturity)

2.1 Distributed Tracing with OpenTelemetry & Tempo

- **Current State**: Request tracking relies on MDC log patterns and manual correlation ID parsing.
- **Recommendation**: Integrate **Micrometer Tracing** with OpenTelemetry exporters to stream distributed trace spans to Grafana Tempo or Jaeger, giving operations teams visual flame graphs of request latencies across all microservice hops.

2.2 Dead Letter Queues (DLQ) & Message Retry Policies

- **Current State**: Unhandled consumer exceptions in RabbitMQ queues could result in message requeuing loops or message drops.
- **Recommendation**: Configure a Dead Letter Exchange (`dlx.aerobook`) and Dead Letter Queues (`booking-created.dlq`) with exponential backoff retries (3 attempts) before routing unprocessable events to an alert queue.

2.3 Centralized Secret Management

- **Current State**: Configuration properties allow environment variable injection, but defaults exist in `application.properties` .
- **Recommendation**: Store production credentials, Razorpay secret keys, and JWT keys in **HashiCorp Vault** or AWS Secrets Manager, injected dynamically into container pods at runtime.

---

3. Long-Term Architectural Goals

3.1 CQRS (Command Query Responsibility Segregation)

- Separate the high-volume Flight Search read models from the transactional Flight Booking write engine, replicating read data to Elasticsearch or MongoDB for sub-millisecond faceted search.

3.2 WebSocket / Server-Sent Events (SSE) for Real-Time Flight Updates

- Replace polling on the passenger and staff dashboards with reactive Server-Sent Events from `notification-service` , instantly updating departure gate numbers and flight delays on passenger mobile screens.

CHAPTER 21

Definitive Platform Service Inventory

This document provides the authoritative inventory of all microservices, frontend applications, and supporting platform components comprising the AeroBook ecosystem.

1. Master Service Inventory Table

Service Name	Directory Path	Port	Technology Stack	Database / Storage	Key Responsibilities	Primary Dependencies	Health Endpoint
Service Discovery	`/service-discovery`	8761	Java 21, Spring Boot 3.3.4, Netflix Eureka Server	In-Memory Registry	Dynamic microservice registry, instance heartbeats, and cluster topology	Standalone	`/actuator/health`
Config Server	`/config-server`	8080	Java 21, Spring Boot 3.3.4, Spring Cloud Config	File / Git Native Profile	Centralized external configuration repository	Standalone	`/actuator/health`
API Gateway	`/api-gateway`	8083	Java 21, Spring Cloud Gateway, WebFlux, Netty	Reactive Non-Blocking	Single entry ingress, routing, JWT perimeter security, header mutation, CORS	Eureka:8761	`/api/gateway/health`
Auth Service	`/auth-service`	8082	Java 21, Spring Boot 3.3.4, Spring Security, JJWT	MySQL: `aerobook_auth_db`	User authentication, credential store, BCrypt hashing, JWT generation, refresh tokens, OTP	Eureka:8761, MySQL:3306, Notification:8091	`/api/auth/health`
User Service	`/user-service`	8084	Java 21, Spring Boot 3.3.4, Spring Data JPA	MySQL: `aerobook_user_db`	Customer profiles, demographic records, airport staff incident reporting	Eureka:8761, MySQL:3306	`/api/v1/users/health`
Flight Service	`/flight-service`	8087	Java 21, Spring Boot 3.3.4, RabbitMQ AMQP	MySQL: `aerobook_flight_db`	Commercial flight schedules, aircraft fleet, seat inventory, search engine	Eureka:8761, MySQL:3306, RabbitMQ:5672	`/api/flights/health`
Fare Service	`/fare-service`	8089	Java 21, Spring Boot 3.3.4, OpenFeign	MySQL: `aerobook_fare_db`	Tiered cabin pricing (Economy, Business, First), GST tax rules, promo coupon validation	Eureka:8761, MySQL:3306, Flight:8087	`/api/fares/health`
Booking Service	`/booking-service`	8088	Java 21, Spring Boot 3.3.4, OpenFeign, RabbitMQ	MySQL: `aerobook_booking_db`	Flight reservation orchestration, PNR generation, passenger manifest, cancellations	Eureka:8761, MySQL:3306, Flight:8087, Fare:8089, RabbitMQ:5672	`/api/bookings/health`
Check-In Service	`/check-in-service`	8090	Java 21, Spring Boot 3.3.4, Spring Data JPA	MySQL: `aerobook_checkin_db`	Passenger check-in, seat confirmation, boarding pass generation, departure gate scanner	Eureka:8761, MySQL:3306, Notification:8091	`/api/check-ins/health`

Service Name	Directory Path	Port	Technology Stack	Database / Storage	Key Responsibilities	Primary Dependencies	Health Endpoint
Notification Service	`/notification-service`	8091	Java 21, Spring Boot 3.3.4, OpenPDF, JavaMail	MySQL: `aerobook_notification_db`	Transactional email dispatch, vector PDF e-tickets/boarding passes, in-app notification inbox	Eureka:8761, MySQL:3306, SMTP Host:587	`/api/notifications/health`
Payment Service	`/payment-service`	8092	Java 21, Spring Boot 3.3.4, Razorpay Java SDK	MySQL: `aerobook_payment_db`	Razorpay checkout orders, HMAC-SHA256 signature verification, webhooks, refunds	Eureka:8761, MySQL:3306, Razorpay Cloud API	`/api/payments/health`
Angular Frontend	`/frontend`	4200	Angular 18.2, TypeScript 5.5, Signals, Standalone	Browser LocalStorage / In-Memory Signals	User interface for passengers, airport staff, and platform administrators	API Gateway:8083	N/A (`/`)

2. Infrastructure & Supporting Components

Component	Default Port	Technology	Purpose
MySQL Server	3306	MySQL Community 8.0+	Relational data persistence hosting 8 isolated microservice schemas
RabbitMQ Broker	5672	RabbitMQ 3.12+ (Erlang OTP)	Asynchronous message broker for decoupled event-driven workflows
RabbitMQ Admin UI	15672	Web Management Plugin	AMQP broker monitoring, queue inspection, and message statistics
SonarQube Server	9000	SonarQube 10.5 Community (Docker)	Automated static code analysis, test coverage, and quality gate governance

CHAPTER 22

Documentation Coverage & Verification Audit

This document audits the completeness, coverage, and verification status of the AeroBook microservices project documentation suite.

1. Documentation Coverage Summary

Area / Component	Expected Count	Documented Count	Coverage Percentage	Verification Status
Backend Microservices	11	11	**100%**	Verified against source code & POMs
Frontend Applications	1	1	**100%**	Verified against Angular 18 source
Relational Databases	8	8	**100%**	Verified against `scripts/seed.sql` & JPA entities
Controllers & Endpoints	13 Controllers	13 Controllers (50+ Endpoints)	**100%**	Verified against `@RequestMapping` code
Message Queues & Events	2 Queues	2 Queues	**100%**	Verified against RabbitMQ config & consumers
Architecture Decision Records	6	6	**100%**	Verified against design implementations
Business Sequence Workflows	7	7	**100%**	Verified against end-to-end service interactions

2. Master Documentation Artifacts Checklist

Core Architecture & Operations Chapters (`docs/`)

- [x] `docs/01-project-overview.md` - Executive summary, problem domain, user personas, requirements
- [x] `docs/02-technology-stack.md` - Complete version matrix, Java 21, Spring Boot 3.3.4, Angular 18, MySQL 8
- [x] `docs/03-system-architecture.md` - High-level system architecture, service topology, routing boundaries
- [x] `docs/04-project-structure.md` - Repository file tree, package anatomy, scripts, build manifests
- [x] `docs/06-frontend.md` - Angular 18 SPA, Standalone components, Signals, `ab-*` UI library, theming
- [x] `docs/07-api-reference.md` - Master API catalog covering all 50+ endpoints across the ecosystem
- [x] `docs/08-database-design.md` - Database-per-service pattern, 8 schemas, Mermaid ERD, table DDLs, seed data
- [x] `docs/09-service-communication.md` - OpenFeign sync clients, Eureka resolution, RabbitMQ AMQP messaging
- [x] `docs/10-security.md` - Perimeter JWT, RBAC permissions matrix, BCrypt hashing, setup keys, CORS
- [x] `docs/11-configuration.md` - Centralized config, environment variables catalog, profile hierarchy
- [x] `docs/12-local-development.md` - Prerequisites, automated startup scripts, seed execution, test accounts
- [x] `docs/13-deployment.md` - Docker multi-stage builds, Docker Compose, Kubernetes manifests, shutdown
- [x] `docs/14-testing.md` - JUnit 5, Mockito, slice tests, Karma/Jasmine, JaCoCo XML, SonarQube gates
- [x] `docs/15-observability.md` - SLF4J/Logback, correlation IDs, Actuator probes, Eureka dashboard
- [x] `docs/16-business-workflows.md` - 7 detailed sequence diagrams for end-to-end aviation workflows
- [x] `docs/17-troubleshooting.md` - Diagnostic runbooks for port collisions, Eureka delay, CORS, token expiry
- [x] `docs/18-glossary.md` - Aviation industry terms (PNR, IATA, boarding pass) & cloud architecture terms
- [x] `docs/19-architecture-decisions.md` - Architecture Decision Records (ADRs 01 to 06)
- [x] `docs/20-recommendations.md` - Prioritized roadmap (Flyway, Redis cache, Resilience4j, rate limiting)
- [x] `docs/21-service-inventory.md` - Definitive master service inventory table with ports, stacks, endpoints
- [x] `docs/22-documentation-coverage.md` - Complete verification audit and compliance metrics

Microservices Detailed Reference (`docs/05-microservices/`)

- [x] `docs/05-microservices/service-discovery.md` - Netflix Eureka Server (Port 8761)
- [x] `docs/05-microservices/config-server.md` - Spring Cloud Config Server (Port 8080)
- [x] `docs/05-microservices/api-gateway.md` - Spring Cloud Reactive Gateway & Netty (Port 8083)
- [x] `docs/05-microservices/auth-service.md` - Authentication & Credential Service (Port 8082)
- [x] `docs/05-microservices/user-service.md` - User Profiles & Incident Service (Port 8084)
- [x] `docs/05-microservices/flight-service.md` - Flights & Aircraft Fleet Service (Port 8087)
- [x] `docs/05-microservices/fare-service.md` - Fares & Promotional Coupons Service (Port 8089)
- [x] `docs/05-microservices/booking-service.md` - Booking & Passenger PNR Service (Port 8088)
- [x] `docs/05-microservices/check-in-service.md` - Check-In & Gate Verification Service (Port 8090)
- [x] `docs/05-microservices/notification-service.md` - Outbound Email & PDF Generation Service (Port 8091)
- [x] `docs/05-microservices/payment-service.md` - Razorpay Payment & Refund Service (Port 8092)

3. Publication & Delivery

- **Markdown Documentation**: All 32 markdown documents committed to `docs/` repository tree.
- **Consolidated PDF Artifact**: Rendered with table of contents, vector diagrams, and syntax highlighting at `docs/Complete_Project_Documentation.pdf`.